

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



ĐỒ ÁN **TỐT NGHIỆP ĐẠI HỌC**

ĐỀ TÀI:
PHÁT TRIỂN ỨNG DỤNG WEB QUẢN LÝ BÃI ĐỖ XE

Giảng viên hướng dẫn : TS. Dương Trần Đức

Nhóm sinh viên thực hiện : Hà Gia Minh – B21DCVT288
Đỗ Minh Hiếu – B21DCVT190
Nguyễn Xuân Hiếu – B21DCCN367

Khoá : 2021 – 2026

Hệ : Đại học chính quy

Hà Nội, tháng 12/2025

NHẬN XÉT, ĐÁNH GIÁ, CHO ĐIỂM
(CỦA GIẢNG VIÊN PHẢN BIỆN)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm:(bằng chữ)

Đồng ý/Không đồng ý cho sinh viên bảo vệ trước hội đồng chấm đồ án tốt nghiệp?

Hà Nội, ngày tháng năm 2026

Giảng viên phản biện

NHẬN XÉT, ĐÁNH GIÁ, CHO ĐIỂM
(CỦA GIẢNG VIÊN HƯỚNG DẪN)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm:(bằng chữ)

Đồng ý/Không đồng ý cho sinh viên bảo vệ trước hội đồng chấm đồ án tốt nghiệp?

Hà Nội, ngày tháng năm 202....

Giảng viên hướng dẫn

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến các thầy cô tại Học viện Công nghệ bưu chính viễn thông, những giảng viên trong hơn bốn năm học vừa qua đã truyền đạt cho em những kiến thức, kinh nghiệm quý báu trong quá trình học tập tại trường và tạo điều kiện thuận lợi để chúng em hoàn thành đồ án.

Chúng em xin chân thành cảm ơn thầy Dương Trần Đức, người đã hướng dẫn và đề xuất những giải pháp trong quá trình thực hiện và giúp đỡ để chúng em có thể hoàn thiện đồ án đúng tiến độ.

Chúng em cũng xin gửi lời cảm ơn tới các anh chị khóa trên, các bạn cũng như các em khóa sau trong Khoa Công nghệ thông tin 1 đã tạo ra một môi trường học tập, rèn luyện tích cực để chúng em có thể phát triển như hôm nay.

Cuối cùng, chúng em xin cảm ơn gia đình, những người đã luôn ở bên cạnh chúng em những lúc khó khăn, giúp đỡ cũng như tạo điều kiện để chúng em có cơ hội được học tập tại Học viện trong suốt thời gian vừa qua.

Do thời gian, kiến thức và kinh nghiệm còn hạn chế nên đồ án chắc chắn còn nhiều điều thiếu sót, chúng em mong muốn được nhận những đóng góp đến từ các thầy cô giúp cho sản phẩm có cơ hội được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

NHÓM TÁC GIẢ

Hà Gia Minh

Đỗ Minh Hiếu

Nguyễn Xuân Hiếu

Mục Lục

LỜI CẢM ƠN.....	5
PHẦN MỞ ĐẦU	7
1. Tổng quan bài toán	7
2. Lý do chọn đề tài	7
3. Mục tiêu đề tài	7
4. Phạm vi đồ án	7
5. Cấu trúc đồ án.....	9
6. Phân công công việc	9
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	10
1.1. Giới thiệu	10
1.2. Các công nghệ và công cụ sử dụng	11
1.3. Kiến trúc	19
CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG	20
2.1. Sơ đồ tổng quan usecase của hệ thống	20
2.2. Kịch bản của các chức năng	22
2.3. Biểu đồ hoạt động.....	37
2.4. Biểu đồ tuần tự	42
2.5. Biểu đồ lớp	47
2.6. Lược đồ cơ sở dữ liệu	51
CHƯƠNG 3: CÀI ĐẶT VÀ THỰC NGHIỆM.....	56
3.1. Môi trường cài đặt và Cấu trúc dự án.....	56
3.2. Đánh giá kết quả thực nghiệm.....	59
CHƯƠNG 4: TỔNG KẾT	63
4.1. Kết luận.....	63
4.2. Hướng phát triển trong tương lai.....	64
TÀI LIỆU THAM KHẢO	66

PHẦN MỞ ĐẦU

1. Tổng quan bài toán

Trong bối cảnh tốc độ đô thị hóa diễn ra mạnh mẽ, sự gia tăng đột biến của các phương tiện cá nhân đã gây ra tình trạng khan hiếm nghiêm trọng các điểm đỗ xe tại các đô thị lớn. Tuy nhiên, thực trạng vận hành tại đa số các bãi đỗ xe hiện nay vẫn còn phụ thuộc vào phương thức thủ công, bộc lộ nhiều bất cập. Việc quản lý ra vào bằng vé giấy hoặc ghi chép tay không chỉ gây ùn tắc trong giờ cao điểm mà còn tiềm ẩn rủi ro sai sót và gian lận. Bên cạnh đó, khách hàng thường xuyên rơi vào trạng thái ức chế và lãng phí nhiên liệu khi phải di chuyển lòng vòng tìm chỗ trống. Khâu thanh toán bằng tiền mặt gây bất tiện, đồng thời khiến việc thống kê doanh thu thiếu minh bạch và chậm trễ, làm cho nhà quản lý không nắm bắt được số liệu thực tế (real-time) để đưa ra quyết định kinh doanh. Xuất phát từ những hạn chế đó, việc xây dựng một hệ thống quản lý bãi đỗ xe ứng dụng công nghệ thông tin để tự động hóa toàn diện quy trình vận hành là yêu cầu cấp thiết và mang tính thực tiễn cao.

2. Lý do chọn đề tài

Việc lựa chọn đề tài xây dựng hệ thống quản lý bãi đỗ xe xuất phát từ hai động lực chính:

2.1. Mở rộng khả năng quản lý và tự động hóa

Trong khi hệ thống cũ còn nặng nề về thủ công và phụ thuộc vào con người, giải pháp mới mang đến sự đột phá nhờ ứng dụng Node.js và công nghệ nhận diện biển số (AI). Hệ thống không chỉ tự động hóa hoàn toàn việc Check-in/Check-out qua API Plate Recognizer để loại bỏ sai sót, mà còn giải quyết bài toán quản trị tài chính bằng cách minh bạch hóa dòng tiền qua các báo cáo trực quan. Đặc biệt, với Dashboard thông minh, người quản lý hoàn toàn nắm bắt được tình trạng bãi xe từ xa mà không cần có mặt trực tiếp.

2.2. Tối ưu hóa trải nghiệm người dùng

Để đáp ứng nhu cầu ngày càng cao về sự tiện lợi và tốc độ trong kỷ nguyên số, hệ thống đã giải quyết triệt để bài toán vận hành thông qua ba giải pháp chính. Thứ nhất, tính năng đặt chỗ trực tuyến (Booking Online) được triển khai giúp người dùng chủ động đảm bảo vị trí đỗ xe trước khi đến. Thứ hai, hệ thống đa dạng hóa các phương thức thanh toán không tiền mặt thông qua việc tích hợp cổng VNPay, mang lại sự an toàn và tiện lợi tối đa. Cuối cùng, nhờ áp dụng quy trình xử lý tự động, thời gian gửi và lấy xe được rút ngắn đáng kể, nâng cao trải nghiệm chung cho người sử dụng.

3. Mục tiêu đề tài

Mục tiêu cốt lõi của đồ án là xây dựng và hoàn thiện một giải pháp phần mềm quản lý bãi đỗ xe toàn diện, đáp ứng chặt chẽ cả hai khía cạnh nghiệp vụ và công nghệ.

Về mặt nghiệp vụ, hệ thống thiết lập quy trình vận hành linh hoạt với tỷ lệ phân bổ 30% vị trí cho đặt trước trực tuyến và 70% cho khách vắng lai, đồng thời tự động hóa hoàn toàn logic tính phí, xử lý các trường hợp vắng mặt (No-show) hoặc quá giờ, cũng như trích xuất báo cáo doanh thu chi tiết theo tháng.

Về mặt công nghệ, ứng dụng được phát triển dựa trên kiến trúc Backend vững chắc sử dụng Node.js và Express Framework, kết hợp với giao diện Frontend hiện đại, hiệu năng cao bằng Next.js và TypeScript. Hệ thống quản trị dữ liệu sử dụng MySQL cùng Sequelize ORM để tối ưu hóa truy vấn, song song với việc tích hợp thành công các dịch vụ bên thứ ba thiết yếu như nhận diện biển số (Plate Recognizer), cổng thanh toán (VNPay) và lưu trữ hình ảnh (Cloudinary).

4. Phạm vi đồ án

Đồ án tập trung xây dựng hệ thống quản lý bãi đỗ xe thông minh với 3 phân hệ chính:

4.1. Khách hàng

Đối với khách hàng, hệ thống được thiết kế với mục tiêu cốt lõi là mang lại sự tiện lợi và tính minh bạch tối đa cho người dùng cuối. Cụ thể, chức năng Đặt chỗ (Booking) cho phép người dùng chủ động tìm kiếm và thực hiện giữ chỗ (Reservation) trước khi di chuyển. Bên cạnh đó, tính năng Tra cứu tình trạng cung cấp thông tin về số lượng vị trí trống (Available) theo thời gian thực, giúp người dùng ra quyết định chính xác.

4.2. Nhân viên

Đối với nhân viên, hệ thống tập trung vào việc tự động hóa quy trình kiểm soát và tối ưu hóa phân bổ vị trí. Thứ nhất, tại quy trình Check-in, nhân viên chỉ cần hỗ trợ chụp ảnh xe, hệ thống sẽ tự động tải lên Server, trích xuất dữ liệu biển số qua AI và đối chiếu tức thì với danh sách đặt chỗ, giúp giảm thiểu thao tác thủ công. Thứ hai, hệ thống áp dụng thuật toán phân bổ vị trí thông minh để xử lý các tình huống phát sinh. Trong trường hợp vị trí khách đã đặt trước (Reservation) gặp sự cố hoặc bảo trì, cơ chế an toàn (Fail-safe) sẽ tự động tìm kiếm một slot Online khác, hoặc trích xuất từ quỹ slot Offline để đảm bảo quyền lợi cho khách hàng. Đối với khách vắng lai, hệ thống ưu tiên tìm slot Offline trống; nếu đã kín chỗ, cơ chế Hybrid sẽ được kích hoạt để quét các slot Online. Hệ thống chỉ cho phép 'mượn' các slot Online này nếu chúng trống trong vòng 25 giờ tiếp theo, đảm bảo tuyệt đối không ảnh hưởng đến khách hàng đã đặt trước.

4.3. Quản lý

Đối với quản lý, hệ thống đóng vai trò trung tâm trong việc kiểm soát vận hành và hỗ trợ ra quyết định chiến lược. Về mặt quản trị hạ tầng, hệ thống cho phép quản lý chặt chẽ đội ngũ nhân sự (Staff) và cấu hình linh hoạt trạng thái các vị trí đỗ (Spot). Chức năng Giám sát thời gian thực (Real-time Monitoring) giúp người quản lý nắm bắt chính xác trạng thái hiện tại của từng ô đỗ: Đang đỗ (Occupied), Đã đặt (Reserved) hoặc Trống (Available), đồng thời hỗ trợ tra cứu chi tiết dữ liệu Vé và Đơn đặt chỗ theo ngày cụ thể.

Đặc biệt, hệ thống tích hợp dashboard báo cáo thông minh cung cấp cái nhìn toàn cảnh về hoạt động kinh doanh: từ việc theo dõi doanh thu thực ngay tại thời điểm hiện tại, đến các biểu đồ phân tích xu hướng doanh thu theo tháng và cơ cấu phương tiện (Ô tô/Xe máy) để điều chỉnh quy hoạch. Ngoài ra, biểu đồ lưu lượng (Traffic Flow) giúp phân tích mật độ xe ra vào theo khung giờ, làm cơ sở quan trọng để bố trí nhân sự hiệu quả trong các giờ cao điểm."

5. Cấu trúc đồ án

Nội dung của đồ án được xây dựng thành các chương như sau:

Chương 1. Cơ sở lý thuyết

Chương 2. Phân tích thiết kế hệ thống

Chương 3. Phát triển ứng dụng

6. Phân công công việc

Hà Gia Minh (Nhóm trưởng)	Nguyễn Xuân Hiếu	Đỗ Minh Hiếu
- Xây dựng backend: User, Staff - Thiết kế database - Viết tài liệu	- Xây dựng backend: admin - Vẽ biểu đồ tuần tự	- Xây dựng frontend cho toàn bộ hệ thống. - Viết cơ sở lý thuyết và cấu trúc của frontend.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Giới thiệu

1.1.1. Tổng quan về Hệ thống Bãi đỗ xe (Parking System)

1.1.1.1 Định nghĩa và đặc điểm cơ bản

Hệ thống bãi đỗ xe (Parking System) là giải pháp ứng dụng công nghệ thông tin và tự động hóa vào việc vận hành, quản lý các bãi trông giữ xe. Hệ thống là sự kết hợp chặt chẽ giữa phần cứng (camera nhận diện, barie, cảm biến) và phần mềm (website, ứng dụng quản lý, cơ sở dữ liệu), nhằm tối ưu hóa quy trình ra vào và sử dụng không gian đỗ xe.

Khác với mô hình bãi xe truyền thống vận hành thủ công (ghi vé giấy, thu tiền mặt), bãi đỗ xe thông minh sở hữu các đặc tính kỹ thuật nổi bật:

Tự động hóa quy trình: Sử dụng công nghệ nhận diện biển số (LPR - License Plate Recognition) để tự động kiểm soát xe ra vào, giảm thiểu sự can thiệp của con người.

Quản lý thời gian thực: Cập nhật trạng thái chỗ trống (slot) ngay lập tức, cho phép người dùng biết chính xác tình trạng bãi xe thông qua giao diện phần mềm.

Tính năng đặt chỗ trước: Người dùng có thể tìm kiếm và giữ chỗ đỗ xe từ xa, đảm bảo có vị trí đỗ khi đến nơi.

Thanh toán điện tử: Tích hợp các cổng thanh toán trực tuyến (VNPay)

1.1.1.2. Tính cấp thiết và vai trò thực tiễn

Trong bối cảnh đô thị hóa nhanh chóng, số lượng phương tiện cá nhân gia tăng mạnh mẽ gây áp lực lớn lên hạ tầng giao thông. Các bãi đỗ xe truyền thống có nhiều hạn chế như: ùn tắc tại cửa soát vé, thất thoát doanh thu do gian lận, và khó khăn trong việc thống kê báo cáo.

Việc triển khai hệ thống bãi đỗ xe thông minh mang lại ý nghĩa thiết thực trên hai khía cạnh:

Hệ thống giúp người quản lý cần kiểm soát chặt chẽ lưu lượng xe vào/ra, loại bỏ nạn vé giả và thất thoát tài chính, tiết kiệm chi phí nhân sự vận hành nhờ cơ chế tự động hóa, cung cấp các báo cáo thống kê trực quan về doanh thu và hiệu suất khai thác bãi xe để hỗ trợ ra quyết định kinh doanh.

Hệ thống giúp khách hàng tiết kiệm thời gian tìm kiếm chỗ đỗ và thời gian chờ đợi tại cổng soát vé.

1.1.1.3. Xu hướng phát triển công nghệ

Lĩnh vực bãi đỗ xe thông minh đang chuyển dịch mạnh mẽ nhờ sự hỗ trợ của các công nghệ nền tảng hiện đại:

Trí tuệ nhân tạo (AI & Computer Vision): Nâng cao độ chính xác trong việc nhận diện biển số xe trong các điều kiện môi trường phức tạp đồng thời giám sát an ninh bãi xe.

Internet of Things (IoT): Kết nối các thiết bị ngoại vi và server để truyền tải dữ liệu trạng thái chỗ đỗ theo thời gian thực (Real-time Communication) thông qua các giao thức như WebSocket.

1.2. Các công nghệ và công cụ sử dụng

1.2.1. Next.js Framework

Next.js là một framework JavaScript mã nguồn mở, được xây dựng dựa trên thư viện React, nhằm mục tiêu đơn giản hóa quá trình phát triển ứng dụng web hiện đại. Framework này hỗ trợ đầy đủ các mô hình render như Client-side Rendering (CSR), Server-side Rendering (SSR) và Static Site Generation (SSG), giúp lập trình viên linh hoạt lựa chọn giải pháp phù hợp với từng loại dự án.

Một trong những điểm mạnh nổi bật của Next.js là khả năng tối ưu hiệu năng và SEO. Nhờ cơ chế render phía server và tạo trang tĩnh sẵn, nội dung website có thể được tải nhanh hơn và dễ dàng được các công cụ tìm kiếm như Google thu thập dữ liệu. Điều này đặc biệt quan trọng đối với các website thương mại điện tử, blog, landing page hoặc các hệ thống cần thứ hạng tìm kiếm cao.

Next.js cung cấp hệ thống routing dựa trên cấu trúc thư mục, giúp việc quản lý và phát triển các trang trở nên đơn giản, trực quan hơn so với React thuần. Ngoài ra, framework còn hỗ trợ API Routes, cho phép xây dựng backend đơn giản ngay trong cùng một dự án, giúp giảm sự phụ thuộc vào các dịch vụ backend bên ngoài.

Bên cạnh đó, Next.js tích hợp nhiều tính năng hiện đại như: tự động chia nhỏ mã nguồn (Automatic Code Splitting) giúp giảm thời gian tải trang, hỗ trợ TypeScript mặc định, tối ưu hình ảnh (Image Optimization), hỗ trợ CSS Module, Sass và Tailwind CSS, incremental Static Regeneration (ISR) cho phép cập nhật nội dung tĩnh mà không cần build lại toàn bộ website.

Với hệ sinh thái mạnh mẽ và khả năng triển khai dễ dàng thông qua nền tảng Vercel, Next.js hiện đang được nhiều công ty và dự án lớn tin dùng như Netflix, TikTok, GitHub và Shopify.

Nhờ những ưu điểm trên, Next.js đã và đang trở thành một trong những framework hàng đầu cho việc phát triển các ứng dụng web hiện đại, hiệu suất cao và dễ mở rộng.

Lý do chúng em lựa chọn cho đề tài là:

Cấu trúc rõ ràng: Next.js cung cấp sẵn cấu trúc định tuyến (Routing) dựa trên hệ thống file, giúp việc phân chia các trang chức năng (Trang chủ, Trang đặt chỗ, Trang quản trị Admin) trở nên khoa học và dễ quản lý hơn so với React thuần. Next.js hỗ trợ **Hiệu**

năng cao: Hỗ trợ Server-Side Rendering và Static Site Generation, giúp trang web tải nhanh hơn, mang lại trải nghiệm mượt mà cho người dùng khi tìm kiếm bãi đỗ xe.

Hỗ trợ API Routes: Dễ dàng tích hợp với Backend Node.js để xử lý dữ liệu.

1.2.2. Ngôn ngữ TypeScript

TypeScript là một ngôn ngữ lập trình mã nguồn mở do Microsoft phát triển, được giới thiệu lần đầu vào năm 2012. TypeScript được thiết kế như một siêu tập hợp (superset) của JavaScript, nghĩa là mọi mã JavaScript hợp lệ đều có thể chạy trong TypeScript. Điểm khác biệt cốt lõi của TypeScript là việc bổ sung hệ thống kiểu tĩnh (static typing), giúp kiểm soát kiểu dữ liệu chặt chẽ hơn trong quá trình phát triển phần mềm.

Nhờ hệ thống kiểu tĩnh, TypeScript cho phép phát hiện sớm các lỗi phổ biến như truyền sai kiểu dữ liệu, truy cập thuộc tính không tồn tại hoặc gọi hàm với tham số không hợp lệ ngay từ giai đoạn biên dịch (compile-time). Điều này giúp giảm thiểu lỗi phát sinh khi ứng dụng đã được triển khai, đặc biệt quan trọng đối với các hệ thống lớn và có nhiều luồng xử lý phức tạp.

TypeScript hỗ trợ nhiều khái niệm lập trình hiện đại như Interface, Type Alias, Enum, Generics, Union Type và Intersection Type, cho phép mô hình hóa dữ liệu và cấu trúc hệ thống một cách rõ ràng, chặt chẽ. Trong các ứng dụng thực tế như hệ thống bãi đỗ xe, các đối tượng như người dùng, vé xe, chỗ đỗ hay giao dịch thanh toán có mối quan hệ phức tạp; việc định nghĩa kiểu dữ liệu cụ thể giúp mã nguồn dễ hiểu, dễ kiểm soát và hạn chế lỗi logic.

Ngoài ra, TypeScript tương thích tốt với các framework và thư viện phổ biến như Next.js, React, Angular và Node.js, đồng thời được tích hợp sâu với các IDE hiện đại như Visual Studio Code. Tính năng IntelliSense của TypeScript cung cấp khả năng tự động gợi ý mã nguồn, kiểm tra lỗi tức thời và hỗ trợ refactor, từ đó giúp tăng năng suất lập trình và giảm thời gian phát triển.

Mặc dù mã TypeScript không thể chạy trực tiếp trên trình duyệt, nhưng nó được biên dịch (transpile) sang JavaScript chuẩn, đảm bảo khả năng tương thích trên mọi môi trường và trình duyệt hiện nay. Nhờ đó, TypeScript vừa mang lại sự an toàn và chặt chẽ trong quá trình phát triển, vừa giữ được tính linh hoạt và phổ biến của JavaScript.

Với những ưu điểm về kiểm soát lỗi, khả năng mở rộng, dễ bảo trì và hỗ trợ công cụ phát triển mạnh mẽ, TypeScript đã trở thành lựa chọn hàng đầu trong việc xây dựng các ứng dụng web quy mô lớn và phức tạp, đặc biệt là các hệ thống yêu cầu độ ổn định và chính xác cao.

Lý do chúng em lựa chọn cho đề tài:

Kiểm soát lỗi chặt chẽ: Hệ thống bãi đỗ xe có các luồng dữ liệu phức tạp (thông tin vé, trạng thái Reservation, dữ liệu thanh toán). TypeScript giúp phát hiện các lỗi sai kiểu dữ liệu

liệu ngay trong quá trình viết code (compile-time) thay vì chờ đến khi chạy chương trình (runtime), giảm thiểu bug đáng kể.

Dễ bảo trì và mở rộng: Việc định nghĩa rõ ràng các Interface và Type cho các đối tượng (như User, Ticket, Spot) giúp code trở nên tường minh, dễ đọc hiểu và dễ dàng nâng cấp tính năng sau này.

1.2.3. Ngôn ngữ Tailwind CSS

Tailwind CSS là một framework CSS theo hướng utility-first, được phát triển nhằm hỗ trợ xây dựng giao diện người dùng một cách nhanh chóng, linh hoạt và nhất quán. Khác với các phương pháp CSS truyền thống hoặc các framework như Bootstrap, Tailwind không cung cấp sẵn các component giao diện cố định mà tập trung vào việc cung cấp các lớp tiện ích (utility classes) nhỏ gọn, mỗi lớp đảm nhiệm một chức năng cụ thể như màu sắc, khoảng cách (margin, padding), kích thước, căn chỉnh, kiểu chữ và hiệu ứng hiển thị.

Cách tiếp cận utility-first cho phép lập trình viên xây dựng giao diện trực tiếp trong mã JSX/TSX thông qua việc kết hợp các lớp CSS có sẵn, từ đó giảm thiểu nhu cầu viết các file CSS riêng biệt. Điều này giúp hạn chế tình trạng CSS dư thừa, tránh xung đột giữa các class và tăng khả năng tái sử dụng giao diện trong toàn bộ hệ thống.

Tailwind CSS hỗ trợ mạnh mẽ cho responsive design, cho phép định nghĩa giao diện tương thích với nhiều kích thước màn hình thông qua các tiền tố như sm, md, lg, xl. Ngoài ra, framework còn hỗ trợ dark mode, trạng thái tương tác (hover, focus, active) và các hiệu ứng chuyển động, giúp nâng cao trải nghiệm người dùng.

Một ưu điểm quan trọng khác của Tailwind CSS là khả năng tùy biến cao thông qua file cấu hình. Nhà phát triển có thể định nghĩa bảng màu, font chữ, khoảng cách và các tiêu chuẩn thiết kế riêng, từ đó đảm bảo tính nhất quán trong toàn bộ giao diện của hệ thống. Khi kết hợp với cơ chế PurgeCSS, Tailwind chỉ giữ lại các lớp CSS thực sự được sử dụng trong quá trình build, giúp giảm đáng kể kích thước file CSS cuối cùng và cải thiện hiệu năng tải trang.

Trong bối cảnh hệ thống bãi đỗ xe, Tailwind CSS giúp xây dựng giao diện quản lý, đặt chỗ và thanh toán một cách nhanh chóng, rõ ràng và dễ mở rộng. Nhờ sự kết hợp chặt chẽ với các framework như Next.js và TypeScript, Tailwind CSS góp phần tạo nên một hệ thống giao diện hiện đại, hiệu suất cao và dễ bảo trì.

Lý do chúng em lựa chọn cho đề tài:

Tăng tốc độ phát triển giao diện: Giảm thời gian viết và quản lý CSS thủ công.

Giao diện nhất quán và dễ bảo trì: Các lớp tiện ích được chuẩn hóa sẵn.

Dễ dàng kết hợp với Next.js: Hoạt động tốt trong môi trường component-based và hỗ trợ responsive design hiệu quả.

1.2.4. Node.js và Express.js

Node.js là một môi trường thực thi JavaScript phía máy chủ (server-side runtime environment), được xây dựng dựa trên V8 JavaScript Engine của Google. Node.js cho phép chạy JavaScript ngoài trình duyệt, mở ra khả năng xây dựng các ứng dụng backend hiệu năng cao. Một trong những đặc điểm quan trọng nhất của Node.js là mô hình xử lý bất đồng bộ và không chặn (asynchronous, non-blocking I/O), giúp hệ thống có thể xử lý nhiều yêu cầu đồng thời mà không cần tạo ra nhiều luồng (thread) như các nền tảng truyền thống.

Nhờ kiến trúc hướng sự kiện (event-driven architecture), Node.js đặc biệt phù hợp với các hệ thống có tần suất truy cập cao, yêu cầu phản hồi nhanh và xử lý nhiều kết nối cùng lúc, chẳng hạn như hệ thống đặt chỗ, thanh toán trực tuyến hoặc quản lý bãi đỗ xe theo thời gian thực. Ngoài ra, hệ sinh thái npm (Node Package Manager) cung cấp hàng trăm nghìn thư viện hỗ trợ, giúp rút ngắn đáng kể thời gian phát triển và triển khai ứng dụng.

Express.js là một framework web tối giản và linh hoạt, được xây dựng trên nền tảng Node.js. Express.js cung cấp các chức năng cốt lõi để xây dựng ứng dụng backend và RESTful API, bao gồm quản lý routing, xử lý request/response, middleware và cơ chế mở rộng linh hoạt. Nhờ thiết kế đơn giản, Express.js cho phép lập trình viên dễ dàng tổ chức cấu trúc dự án, kiểm soát luồng xử lý nghiệp vụ và tích hợp các thư viện bên thứ ba như xác thực người dùng, logging, bảo mật và xử lý lỗi.

Lý do chúng em lựa chọn: So với các nền tảng khác như Django (Python) hay Spring Boot (Java), Node.js được lựa chọn nhờ sự thống nhất về ngôn ngữ lập trình (JavaScript) giữa frontend và backend. Điều này giúp rút ngắn thời gian phát triển, dễ dàng tích hợp và tận dụng được cộng đồng hỗ trợ lớn.

1.2.5. Kiến trúc RESTful API

RESTful API (Representational State Transfer) là một kiểu kiến trúc phổ biến được sử dụng để xây dựng các dịch vụ web, cho phép các hệ thống khác nhau giao tiếp và trao đổi dữ liệu thông qua giao thức HTTP. Trong kiến trúc này, mọi thành phần dữ liệu trong hệ thống được xem như các tài nguyên (resource) và được định danh duy nhất thông qua các URL (endpoint) rõ ràng, dễ hiểu.

Cơ chế hoạt động của RESTful API dựa trên các phương thức HTTP tiêu chuẩn như GET (lấy dữ liệu), POST (tạo mới tài nguyên), PUT/PATCH (cập nhật tài nguyên) và DELETE (xóa tài nguyên). Mỗi yêu cầu từ phía client (Frontend) gửi lên server (Backend) đều là stateless, tức là server không lưu trạng thái của client giữa các lần request. Điều này giúp hệ thống dễ dàng mở rộng và xử lý đồng thời nhiều yêu cầu từ người dùng.

Một ưu điểm quan trọng của kiến trúc REST là tính tách biệt giữa giao diện và dữ liệu. Frontend chỉ đóng vai trò hiển thị và tương tác với người dùng, trong khi Backend chịu

trách nhiệm xử lý logic nghiệp vụ và quản lý dữ liệu. Nhờ sự phân tách này, các thành phần của hệ thống có thể được phát triển, nâng cấp hoặc thay thế một cách độc lập mà không ảnh hưởng lớn đến toàn bộ hệ thống.

Bên cạnh đó, RESTful API thường sử dụng định dạng dữ liệu JSON, giúp dữ liệu nhẹ, dễ đọc và dễ xử lý trên nhiều nền tảng khác nhau. Điều này cho phép hệ thống dễ dàng tích hợp với các ứng dụng khác như ứng dụng di động, hệ thống quản lý bên thứ ba hoặc các dịch vụ phân tích dữ liệu trong tương lai.

1.2.6. Hệ quản trị MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) mã nguồn mở, được sử dụng rộng rãi trong các hệ thống thông tin hiện đại. MySQL cho phép tổ chức và lưu trữ dữ liệu dưới dạng các bảng có cấu trúc chặt chẽ, với các mối quan hệ được xác định thông qua khóa chính (Primary Key) và khóa ngoại (Foreign Key), từ đó đảm bảo tính nhất quán, toàn vẹn và độ tin cậy của dữ liệu.

Trong hệ thống, MySQL đảm nhiệm vai trò lưu trữ toàn bộ dữ liệu. Nhờ khả năng xử lý truy vấn hiệu quả, MySQL đáp ứng tốt các yêu cầu về truy xuất dữ liệu nhanh, chính xác và ổn định ngay cả khi khối lượng dữ liệu tăng lên.

MySQL hỗ trợ đầy đủ các tính năng quan trọng như giao dịch (transaction), ràng buộc toàn vẹn dữ liệu, chỉ mục (index) và cơ chế khóa, giúp đảm bảo dữ liệu luôn ở trạng thái nhất quán trong các tình huống có nhiều thao tác đồng thời. Điều này đặc biệt quan trọng đối với các hệ thống yêu cầu độ chính xác cao, chẳng hạn như quản lý vé, hóa đơn, thông tin khách hàng.

Việc kết hợp MySQL với Sequelize ORM trong môi trường Node.js giúp đơn giản hóa quá trình thao tác với cơ sở dữ liệu. Sequelize cho phép ánh xạ các bảng dữ liệu thành các đối tượng trong mã nguồn, giúp lập trình viên thao tác dữ liệu bằng cú pháp hướng đối tượng thay vì viết các câu lệnh SQL phức tạp. Cách tiếp cận này giúp giảm thiểu lỗi logic, tăng khả năng tái sử dụng mã nguồn và đáp ứng tốt yêu cầu phi chức năng về dễ bảo trì và mở rộng hệ thống.

Lý do chúng em chọn cho đề tài: Dữ liệu bãi đỗ xe (Vé, Hóa đơn, Thông tin khách hàng) có cấu trúc chặt chẽ, cần tính nhất quán cao và quan hệ rõ ràng nên MySQL phù hợp hơn NoSQL (như MongoDB).

1.2.7. Thư viện Sequelize (ORM)

Sequelize là một Object–Relational Mapping (ORM) hiện đại, được phát triển dành cho môi trường Node.js. Thư viện này cho phép ánh xạ các bảng trong cơ sở dữ liệu quan hệ thành các đối tượng trong chương trình, giúp lập trình viên thao tác dữ liệu thông qua cú pháp hướng đối tượng thay vì phải viết trực tiếp các câu lệnh SQL phức tạp. Sequelize hỗ trợ nhiều hệ quản trị cơ sở dữ liệu phổ biến như MySQL, PostgreSQL, MariaDB,

SQLite, SQL Server và Oracle, đồng thời có khả năng hoạt động trên nhiều hệ điều hành khác nhau.

Một trong những ưu điểm nổi bật của Sequelize là khả năng quản lý quan hệ giữa các bảng thông qua các kiểu liên kết như One-to-One, One-to-Many và Many-to-Many. Điều này giúp mô hình hóa các thực thể trong hệ thống (ví dụ: User, Reservation, Spot, Ticket) một cách rõ ràng và nhất quán. Ngoài ra, Sequelize hỗ trợ eager loading và lazy loading, cho phép linh hoạt trong việc truy xuất dữ liệu liên quan, tối ưu hiệu năng cho từng trường hợp sử dụng.

Sequelize cũng cung cấp cơ chế transaction mạnh mẽ, giúp đảm bảo tính toàn vẹn dữ liệu trong các nghiệp vụ quan trọng như đặt chỗ, thanh toán hoặc cập nhật trạng thái vé. Bên cạnh đó, ORM này hỗ trợ migration và seeding, giúp quản lý phiên bản cơ sở dữ liệu, dễ dàng thay đổi cấu trúc bảng và khởi tạo dữ liệu mẫu trong quá trình phát triển và triển khai hệ thống.

Việc sử dụng Sequelize giúp giảm thiểu các rủi ro bảo mật phổ biến như SQL Injection, do các truy vấn được xây dựng thông qua các phương thức an toàn đã được kiểm soát. Đồng thời, mã nguồn trở nên ngắn gọn, dễ đọc và dễ bảo trì hơn so với việc sử dụng SQL thuần, đặc biệt trong các dự án có quy mô vừa và lớn.

Trong hệ thống, Sequelize được ứng dụng để quản lý và thao tác dữ liệu cho các bảng như người dùng, vé, đặt chỗ và vị trí đỗ xe. Nhờ khả năng tích hợp chặt chẽ với Node.js và Express.js, Sequelize góp phần xây dựng một tầng truy cập dữ liệu ổn định, an toàn và dễ mở rộng, đáp ứng tốt các yêu cầu chức năng và phi chức năng của đề tài.

Ứng dụng trong đồ án: Giúp thao tác với database bằng mã JavaScript (Model, Find, Create) thay vì viết câu lệnh SQL trần. Giúp code gọn hơn, tránh lỗi SQL Injection và dễ dàng quản lý các bảng (User, Reservation, Spot).

1.2.7. Công nghệ giao tiếp thời gian thực (WebSocket)

WebSocket là một giao thức truyền thông cho phép thiết lập kết nối hai chiều (full-duplex) giữa client và server thông qua một kết nối duy nhất và được duy trì liên tục. Khác với mô hình HTTP truyền thống – nơi client phải liên tục gửi request để kiểm tra dữ liệu mới (polling) – WebSocket cho phép server chủ động gửi dữ liệu đến client ngay khi có sự thay đổi xảy ra, từ đó đáp ứng hiệu quả các yêu cầu giao tiếp thời gian thực.

Trong hệ thống, WebSocket được triển khai thông qua thư viện Socket.io, giúp đơn giản hóa quá trình thiết lập và quản lý kết nối thời gian thực giữa frontend và backend. Socket.io cung cấp các cơ chế như tự động kết nối lại, quản lý phòng (room), sự kiện (event) và xử lý lỗi, giúp hệ thống hoạt động ổn định ngay cả trong điều kiện mạng không ổn định.

Ứng dụng nổi bật của WebSocket trong đồ án là cập nhật trạng thái bãi đỗ xe theo thời gian thực. Khi một xe thực hiện thao tác check-in hoặc check-out thành công, backend

sẽ phát (emit) một sự kiện thông qua WebSocket đến các client đang kết nối. Ngay lập tức, giao diện người dùng được cập nhật trạng thái chỗ đỗ (Trống/Đầy) mà không cần tải lại trang, từ đó nâng cao trải nghiệm người dùng và đảm bảo thông tin hiển thị luôn chính xác.

So với các giải pháp khác như AJAX polling hoặc long polling, WebSocket giúp giảm độ trễ, tiết kiệm tài nguyên mạng và tăng hiệu năng tổng thể của hệ thống. Điều này đặc biệt quan trọng đối với các hệ thống có nhiều người dùng truy cập đồng thời và yêu cầu cập nhật dữ liệu liên tục, chẳng hạn như hệ thống bãi đỗ xe thông minh, bảng điều khiển thời gian thực hoặc ứng dụng giám sát.

Nhờ khả năng giao tiếp nhanh, ổn định và thời gian thực, WebSocket đóng vai trò quan trọng trong việc xây dựng các tính năng thông minh cho hệ thống. Việc tích hợp WebSocket cùng với Node.js và Express.js giúp hệ thống không chỉ đáp ứng tốt yêu cầu chức năng mà còn nâng cao tính hiện đại và khả năng mở rộng trong tương lai.

Ứng dụng trong đồ án: Đây là tính năng thông minh. Dùng để cập nhật trạng thái bãi đỗ xe (Trống/Đầy) ngay lập tức lên màn hình người dùng mà không cần tải lại trang. Khi xe check-in thành công, Socket.io bắn tín hiệu để client cập nhật giao diện.

1.2.8. Công nghệ nhận diện biển số (PlateRecognizer.com)

Plate Recognizer là một dịch vụ nhận diện biển số xe tự động (Automatic License Plate Recognition – ALPR) dựa trên trí tuệ nhân tạo (AI) và học máy (Machine Learning). Dịch vụ này cho phép trích xuất thông tin biển số xe từ hình ảnh hoặc video một cách nhanh chóng và chính xác, hỗ trợ nhiều quốc gia và định dạng biển số khác nhau.

Trong hệ thống bãi đỗ xe, Plate Recognizer được sử dụng để tự động nhận diện biển số khi xe check-in/check-out, thay thế cho việc nhập liệu thủ công. Hình ảnh từ camera được gửi đến API của Plate Recognizer, sau đó hệ thống nhận về kết quả nhận diện bao gồm chuỗi biển số, độ tin cậy (confidence score) và các thông tin liên quan khác. Dữ liệu này được dùng để xác định xe, tạo vé, đối chiếu lịch sử ra vào và phục vụ cho các nghiệp vụ thanh toán.

Việc ứng dụng Plate Recognizer giúp: tăng độ chính xác trong quá trình nhận diện xe, giảm thời gian xử lý và sai sót do con người và nâng cao tính tự động hóa và hiện đại của hệ thống bãi đỗ xe

1.2.9. Cổng thanh toán trực tuyến VNPay

VNPay là một trong những cổng thanh toán điện tử phổ biến tại Việt Nam, hỗ trợ nhiều hình thức thanh toán như thẻ ngân hàng nội địa, Internet Banking và ví điện tử. Trong đồ án, VNPay được tích hợp thông qua môi trường Sandbox, cho phép mô phỏng và kiểm thử đầy đủ quy trình thanh toán mà không phát sinh giao dịch thực tế.

Hệ thống sử dụng VNPay để xử lý các nghiệp vụ thanh toán phí gửi xe một cách an toàn và tiện lợi. Khi người dùng thực hiện thanh toán, hệ thống sẽ tạo yêu cầu thanh toán và chuyển hướng người dùng đến cổng VNPay. Sau khi giao dịch hoàn tất, VNPay gửi phản hồi về hệ thống thông qua cơ chế IPN (Instant Payment Notification) hoặc URL callback, giúp backend cập nhật trạng thái hóa đơn (thành công, thất bại, hủy giao dịch) một cách tự động.

Việc tích hợp VNPay mang lại nhiều lợi ích như: đảm bảo tính bảo mật và an toàn trong giao dịch tài chính, tự động hóa quy trình thanh toán và cập nhật hóa đơn và nâng cao trải nghiệm người dùng với các phương thức thanh toán phổ biến.

1.2.10. Bảo mật hệ thống

JWT (JSON Web Token) là một chuẩn mở được sử dụng phổ biến trong việc xác thực (Authentication) và phân quyền (Authorization) trong các hệ thống web hiện đại. JWT cho phép truyền tải thông tin xác thực giữa client và server một cách an toàn dưới dạng một chuỗi JSON đã được mã hóa và ký số.

Một JWT thường bao gồm ba phần chính: Header, Payload và Signature. Header chứa thông tin về thuật toán mã hóa được sử dụng; Payload lưu trữ các thông tin định danh người dùng và quyền truy cập; Signature được tạo ra nhằm đảm bảo tính toàn vẹn của token và ngăn chặn việc giả mạo dữ liệu. Nhờ cơ chế ký số này, server có thể xác minh tính hợp lệ của token mà không cần lưu trữ trạng thái phiên của người dùng.

Trong hệ thống, JWT được sử dụng để xác thực người dùng khi đăng nhập và phân quyền truy cập vào các chức năng quản trị như thêm, sửa hoặc xóa dữ liệu. Sau khi đăng nhập thành công, server sẽ phát hành một token và gửi về cho client. Token này sẽ được đính kèm trong mỗi request tiếp theo (thường thông qua HTTP Header), giúp server xác định danh tính và quyền hạn của người dùng một cách nhanh chóng và an toàn.

Ưu điểm nổi bật của JWT là tính stateless, nghĩa là server không cần lưu trữ thông tin phiên làm việc của người dùng. Điều này giúp hệ thống dễ dàng mở rộng, giảm tải cho máy chủ và đặc biệt phù hợp với kiến trúc RESTful API cũng như các hệ thống phân tán hoặc microservices. So với các phương pháp xác thực truyền thống như session-based authentication, JWT mang lại sự linh hoạt cao hơn trong việc triển khai và tích hợp.

Bên cạnh đó, JWT có thể dễ dàng kết hợp với các cơ chế bảo mật khác như HTTPS, refresh token và kiểm soát thời gian hết hạn (expiration time), từ đó nâng cao mức độ an toàn cho hệ thống. Việc áp dụng JWT giúp đảm bảo rằng mỗi request gửi lên server

đều được xác thực, hạn chế truy cập trái phép và bảo vệ dữ liệu khỏi các nguy cơ tấn công.

Nhờ những ưu điểm về hiệu năng, bảo mật và khả năng mở rộng, JWT là giải pháp phù hợp để xây dựng cơ chế xác thực và phân quyền cho hệ thống, đáp ứng tốt các yêu cầu bảo mật trong đề án.

Ứng dụng: Dùng để xác thực người dùng và phân quyền. Đảm bảo mỗi request gửi lên server đều an toàn và xác định được danh tính.

1.3. Kiến trúc

Hệ thống được thiết kế dựa trên mô hình Client-Server 3 tầng (3-Tier Architecture). Mô hình này giúp tách biệt rõ ràng giữa giao diện người dùng, xử lý nghiệp vụ và lưu trữ dữ liệu, giúp hệ thống dễ dàng bảo trì, nâng cấp và mở rộng.

Hệ thống bao gồm 3 thành phần chính:

1.3.1. Tầng Giao diện

Công nghệ: Next.js (React Framework), TypeScript, TailwindCSS.

Vai trò: tầng giao diện là nơi người dùng (Khách hàng/Nhân viên) tương tác trực tiếp với hệ thống thông qua trình duyệt web, hiển thị thông tin trực quan (trạng thái các slot, thông tin vé), gửi các yêu cầu (Request) đến Server thông qua giao thức HTTP/HTTPS (cho các tác vụ như đăng nhập, đặt chỗ) và giao thức WebSocket (để nhận cập nhật trạng thái bãi xe theo thời gian thực).

1.3.2 Tầng Nghiệp vụ (Business Logic Layer - Server Side)

Công nghệ: Node.js, Express Framework.

Vai trò: tầng nghiệp vụ là trung tâm xử lý của hệ thống, tiếp nhận yêu cầu từ Client, thực hiện các quy trình nghiệp vụ phức tạp, xác thực người dùng (Authentication) và phân quyền (Authorization) bằng JWT, tính toán phí gửi xe, xử lý logic đặt chỗ (Reservation logic), xử lý thanh toán qua cổng VNPay, xử lý dữ liệu hình ảnh biển số xe (tích hợp API nhận diện), sử dụng Socket.io để phát (emit) các tín hiệu thay đổi trạng thái tới tất cả các Client đang kết nối.

1.3.3. Tầng Dữ liệu (Data Access Layer)

Công nghệ: MySQL, Sequelize ORM.

Vai trò: tầng dữ liệu giúp lưu trữ bền vững toàn bộ dữ liệu của hệ thống: Thông tin người dùng, lịch sử xe ra vào, hóa đơn, cấu hình bãi đỗ, đảm bảo tính toàn vẹn và nhất quán của dữ liệu (Data Integrity) và tương tác với tầng nghiệp vụ thông qua thư viện Sequelize ORM, giúp chuẩn hóa các truy vấn và tăng tính bảo mật (tránh SQL Injection).

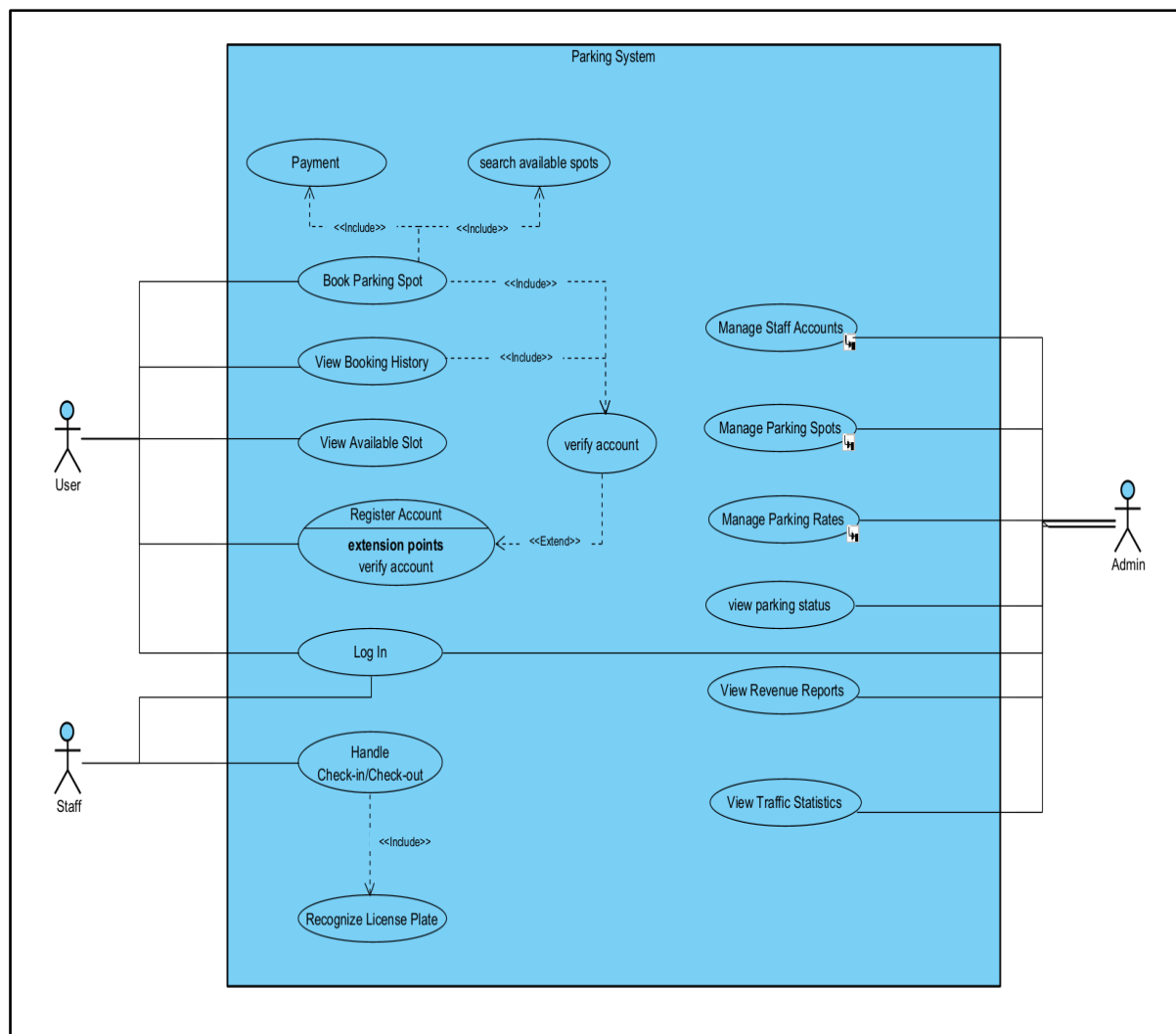
CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. Sơ đồ tổng quan usecase của hệ thống

Tác nhân tham gia vào hệ thống:

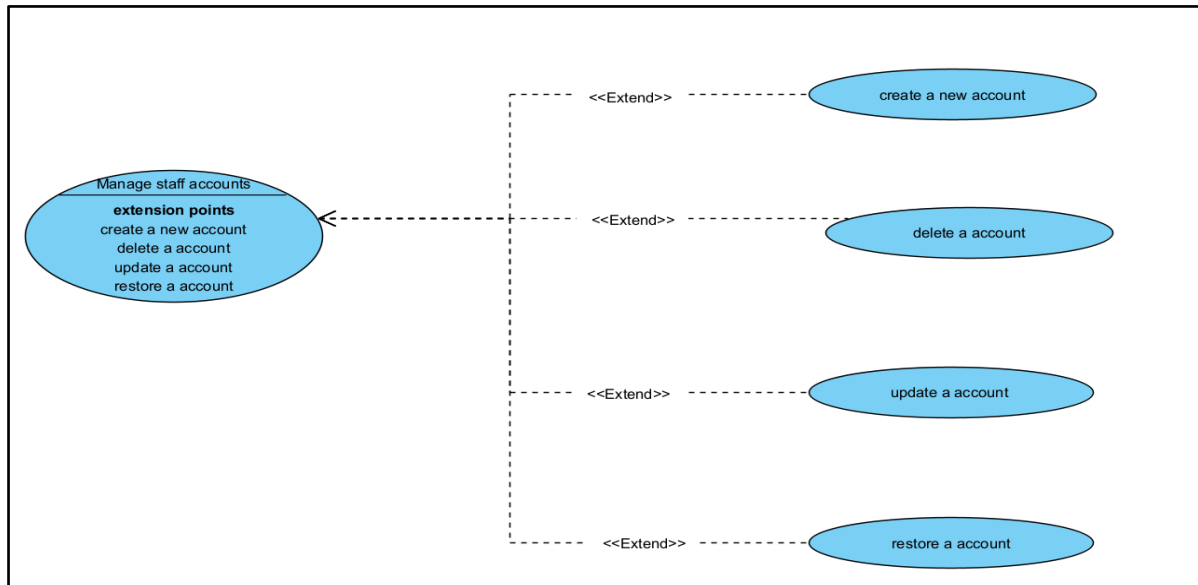
STT	Tác nhân	Mô tả
1	User	Người dùng có thể xem được số lượng chỗ khả thi hiện tại của bãi đỗ và có thể đặt chỗ trước
2	Staff	Nhân viên hỗ trợ vào quá trình check in và check out xe
3	Admin	Quản trị viên quản lý tài khoản nhân viên, vị trí đỗ xe, biểu phí và xem được doanh thu của bãi đỗ xe

Biểu đồ Usecase tổng quát

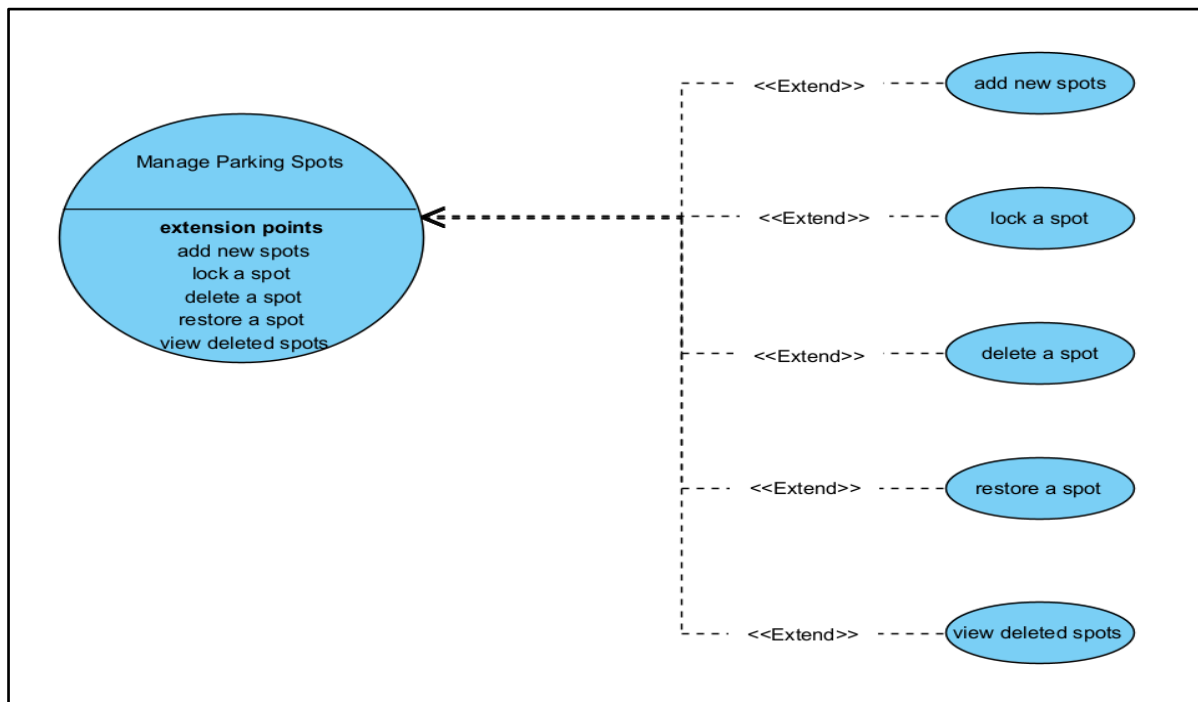


Hình 1: Biểu đồ use case tổng quát

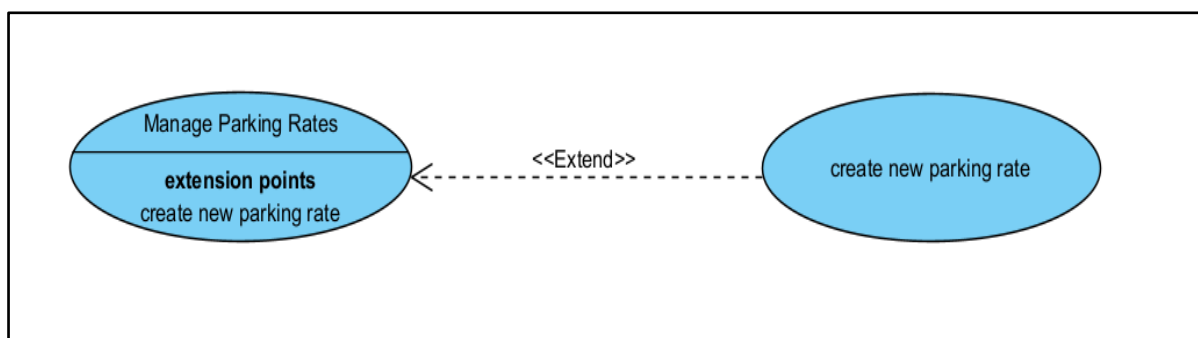
Biểu đồ Usecase chi tiết:



Hình 2: Biểu đồ use case manage staff accounts



Hình 3: Biểu đồ use case manage parking spots



Hình 4: Biểu đồ use case manage parking rates

2.2. Kịch bản của các chức năng

2.2.1. Đăng ký (Register)

Mô tả chức năng: chức năng cho phép người dùng (khách hàng) tạo tài khoản mới để sử dụng dịch vụ đặt chỗ. Hệ thống sử dụng số điện thoại làm tên đăng nhập (username) và yêu cầu xác thực qua email để đảm bảo tính chính xác.

Bảng 1: Kịch bản chức năng Đăng ký

Mã Use Case	UC_01
Đối với khách hàng:	Đăng ký
Đối với khách hàng:	User
Đối với khách hàng:	Người dùng truy cập trang đăng ký Người dùng nhập đầy đủ thông tin bắt buộc: username, password, email, confirm password
Đối với khách hàng:	Tài khoản được tạo thành công và hệ thống gửi link verify đến gmail của người dùng
Chuỗi sự kiện chính: 1. User chọn chức năng Đăng ký 2. Hệ thống hiển thị form đăng ký 3. User nhập các thông tin yêu cầu và nhấn nút Đăng ký 4. Hệ thống kiểm tra tính hợp lệ của dữ liệu (username là sdt, username và gmail không được tồn tại trong hệ thống) 5. Nếu hợp lệ, hệ thống mã hóa mật khẩu và tạo tài khoản mới trong database, thông báo đăng ký thành công và gửi link verify tài khoản ở gmail đăng ký.	
Ngoại lệ: 4. Lỗi dữ liệu đầu vào - Định dạng gmail không hợp lệ hoặc là gmail đã tồn tại → báo lỗi và yêu cầu nhập lại - Định dạng số điện thoại không hợp lệ hay là số điện thoại đã tồn tại → báo lỗi và yêu cầu nhập lại	
Quy tắc nghiệp vụ: - Username: bắt buộc phải là số điện thoại - Mật khẩu trước khi lưu vào database phải mã hoá	

2.2.2. Đăng nhập (Login)

Mô tả chức năng: Chức năng cho phép các tác nhân (Khách hàng, Nhân viên, Admin) xác thực danh tính để truy cập vào hệ thống.

Bảng 2: Kịch bản chức năng Đăng nhập

Mã Use Case	UC_02
Tên usecase	Đăng nhập
Tác nhân chính	Khách hàng (User), Nhân viên (Staff), Quản trị viên (Admin)
Điều kiện tiên quyết	Tài khoản đã tồn tại trong hệ thống
Hậu điều kiện	Truy cập được vào trang chủ
Chuỗi sự kiện chính: 1. Người dùng chọn chức năng Đăng nhập 2. Hệ thống hiển thị form đăng nhập 3. Người dùng nhập username, password rồi nhấn nút Đăng nhập 4. Hệ thống kiểm tra sự tồn tại và trạng thái của tài khoản, so sánh mật khẩu với mật khẩu đã mã hóa trong database 5. Nếu đúng, hệ thống sinh token chứa thông tin người dùng và vai trò, sau đó trả token cho client và điều hướng đến giao diện tương ứng	
Ngoại lệ 4.1. Không tìm thấy tài khoản → thông báo tài khoản không tồn tại 4.2. Mật khẩu sai → thông báo sai mật khẩu 4.3. Tài khoản bị khóa → thông báo tài khoản đã bị khóa	
Quy tắc nghiệp vụ: - Bảo mật: Mật khẩu không bao giờ được gửi về Client, chỉ gửi Token. - Cơ chế Token: Sử dụng JSON Web Token (JWT). Token sẽ hết hạn sau một khoảng thời gian nhất định(1 ngày)	

2.2.3 Đặt chỗ(Reservation)

Mô tả chức năng: Cho phép người dùng tìm kiếm vị trí đỗ xe trống trong khoảng thời gian mong muốn, chọn chỗ và thực hiện thanh toán để giữ chỗ.

Bảng 3: Kịch bản chức năng Đặt chỗ

Mã Use Case	UC_03
Tên usecase	Đặt chỗ
Tác nhân chính	User
Điều kiện tiên quyết	Người dùng đã đăng nhập thành công vào hệ thống Người dùng đã verify tài khoản gmail
Hậu điều kiện	Một bản ghi Reservation được tạo với trạng thái CONFIRMED (nếu thanh toán thành công) hoặc PENDING (chờ thanh toán).
Chuỗi sự kiện chính: 1. Người dùng chọn chức năng đặt chỗ. 2. Hệ thống hiển thị form đặt chỗ. 3. Người dùng nhập ngày, giờ vào, giờ ra, loại xe, biển số. 4. Hệ thống kiểm tra tình trạng trống của các slot trong khoảng thời gian đó, và trả về các spot khả dụng. 5. Người dùng chọn spot và bấm đặt reservation. 6. Hệ thống hiển thị payment trong đó có đường link vnpay để khách thanh toán 7. Người dùng truy cập đường link và thanh toán 8. Hệ thống lắng nghe sự kiện nếu thành công trả về success và không thành công trả về fail	
Ngoại lệ 3.1. Thông tin nhập vào không hợp lệ → Hệ thống yêu cầu nhập lại 4.1. Hết slot trong khung thời gian → Hệ thống thông báo hết chỗ đỗ	

2.2.4. Xem trạng thái chỗ trống (View Real-time Availability)

Mô tả chức năng: Cho phép người dùng (bao gồm cả khách vãng lai chưa đăng nhập) xem danh sách các vị trí đỗ xe đang trống tại thời điểm hiện tại để quyết định có vào bãi hay không.

Bảng 4: Kịch bản chức năng xem trạng thái trống

Mã Use Case	UC_03
Tên usecase	Xem trạng thái chỗ trống (View Real-time Availability)
Tác nhân chính	User
Điều kiện tiên quyết	Hệ thống (Server) đang hoạt động bình thường.
Hậu điều kiện	Người dùng nắm được số lượng chỗ trống trên giao diện.
Chuỗi sự kiện chính: 1. Người dùng truy cập vào trang chủ của Website (không bắt buộc đăng nhập). 2. Hệ thống thực hiện truy vấn dữ liệu thời gian thực	
Quy tắc nghiệp vụ: - Chỗ trống (Available): nghĩa là chỗ đỗ xe cho khách đến vãng lai không có người đỗ hoặc chỗ đỗ dành cho online không có người đặt trong vòng 25h tính từ bây giờ - Real-time: trạng thái được cập nhật liên tục (socket.io)	

2.2.5 Check In

Mô tả chức năng: Chức năng ghi nhận xe vào bãi. Hệ thống sử dụng AI nhận diện biển số để tự động kiểm tra xem khách đã đặt chỗ trước hay chưa. Nếu đã đặt, hệ thống chỉ dẫn vào đúng chỗ đó. Nếu chưa (khách vãng lai), hệ thống tự động tìm chỗ trống và cấp vé.

Bảng 5: Kịch bản chức năng Check in

Mã Use Case	UC_05
Tên usecase	Check in xe
Tác nhân chính	Staff
Điều kiện tiên quyết	- Nhân viên đã đăng nhập và đang ở màn hình Check-in. - Xe ô tô đang dừng trước Camera tại cổng vào. - Camera hoạt động bình thường.

	- Xe chưa có mặt trong bãi (Hệ thống check trạng thái active của Ticket).
Hậu điều kiện	- Tạo mới một Ticket trong database - Trạng thái Spot chuyển thành status = false - (Nếu có Reservation) Trạng thái Reservation chuyển thành CHECKIN - Ảnh check-in được upload lên Cloud và link được cập nhật vào Ticket.
Chuỗi sự kiện chính: 1. Nhân viên chụp ảnh xe và gửi yêu cầu. 2. Hệ thống thực hiện song song: Gửi ảnh sang AI nhận diện và khởi tạo hàm upload ảnh lên Cloud. 3. Hệ thống nhận kết quả từ AI, chuẩn hóa loại xe (Motorcycle -> MOTORBIKE) và biển số. 4. Hệ thống kiểm tra xem xe này có đang ở trong bãi không. Nếu có thì hệ thống báo lỗi (Xem ngoại lệ 4a). 5. Hệ thống tìm kiếm đơn đặt chỗ (Reservation) thỏa mãn: Biển số, Loại xe, Trạng thái CONFIRMED, Thời gian hợp lệ. 6. Phân nhánh xử lý Reservation (Logic thời gian): - Đến muộn quá chỉ còn 10 phút hoặc Đến quá sớm (> 60 phút): chuyển sang xử lý như Khách vắng lai (Bước 8). - Đến sớm trong khoảng 5 - 60 phút: Báo lỗi, yêu cầu khách chờ (Xem ngoại lệ 6a). - Đến đúng giờ (hoặc sớm < 5 phút): Chấp nhận Check-in. 7. Xử lý cho Khách đặt trước (Online): - Kiểm tra Spot đã đặt có hoạt động tốt không. - Nếu Spot lỗi/bảo trì: Hệ thống tự động tìm Spot khác thay thế (Tìm Online nếu hết tìm Offline). - Cập nhật trạng thái Reservation sang CHECKIN. 8. Xử lý cho Khách vắng lai (Offline): - Hệ thống ưu tiên tìm Spot Offline còn trống.	

- Nếu hết Spot Offline -> Tìm Spot Online còn trống (không có reservation trong vòng 25h tính từ thời điểm bây giờ)
 - Nếu hết sạch chỗ -> Báo lỗi (Xem ngoại lệ 8a).
 - Tạo một Reservation "ảo" (Channel: OFFLINE).
9. Hệ thống cập nhật trạng thái Spot(status) thành Bận (False) và tạo Ticket mới.
10. Hệ thống trả về kết quả thành công: Khu vực (Area), Vị trí (Position), Biển số cho nhân viên.
11. (Chạy ngầm) Cập nhật URL ảnh từ Cloudinary vào Ticket sau khi upload xong.

Ngoại lệ

3a. AI không nhận diện được/Lỗi loại xe:

- Trả về lỗi 400: "Không thể xác định biển số/loại xe".

4a. Xe đang ở trong bãi:

- Hệ thống phát hiện Ticket cũ chưa checkout.
- Trả về lỗi 400: "Xe đang ở trong bãi". Rollback giao dịch.

6a. Khách đến quá sớm (từ 5 đến 60 phút):

- Hệ thống tính toán diffMinutes.
- Trả về lỗi 400: "Bạn đến sớm X phút. Giờ đặt chỗ là HH:mm". Yêu cầu khách chờ.

7a. Spot đặt trước bị hỏng và hết chỗ thay thế:

- Hệ thống không tìm được slot nào thay thế cho slot đã đặt bị lỗi.
- Trả về lỗi 404: "Chỗ đỗ đang bảo trì và hết chỗ thay thế".

8a. Bãi xe hết chỗ (Full):

- Khách vắng lai đến nhưng không tìm thấy spot Offline lẫn Online nào trống.
- Trả về lỗi 404: "Bãi xe đã hết chỗ trống".

2.2.6. Check Out

Mô tả chức năng: Chức năng xử lý xe ra khỏi bãi. Hệ thống tính toán thời gian đỗ, đối chiếu với các khoản đã thanh toán trước, tính phí phát sinh (Overtime) và tạo hóa đơn.

Bảng 6: Kịch bản chức năng Check out

Mã usecase	UC_06
Tên usecase	Check out

Tác nhân chính	Staff
Điều kiện tiên quyết	- Xe đang có Ticket ở trạng thái Active trong hệ thống - Bảng giá (Parking Rate) đã được cấu hình trong hệ thống.
Hậu điều kiện	- Ticket chuyển trạng thái sang inactive, cập nhật giờ ra. - Spot được giải phóng (Status: true). - Reservation chuyển sang CHECKOUT
Chuỗi sự kiện chính: 1. Nhân viên chụp ảnh xe tại cổng ra và gửi yêu cầu. 2. Hệ thống nhận diện biển số và loại xe (AI). 3. Hệ thống truy vấn song song (Promise.all): - Lấy bảng giá chuẩn (Standard) và giá quá giờ (Overtime). - Tìm kiếm Ticket đang hoạt động dựa trên biển số. 4. Hệ thống lấy thông tin Reservation và lịch sử Payment (đã thanh toán online) của Ticket đó. 5. Hệ thống tính toán thời gian đỗ: $\text{Duration} = \text{Giờ hiện tại} - \text{Giờ vào}$. 6. Hệ thống tính toán Tổng tiền phải trả (Total Price): 6a. Trường hợp Khách Online (Có Reservation & Payment thành công): - Nếu $\text{Duration} > \text{Thời gian đặt trước}$: $\text{Phí} = (\text{Phí ban đầu}) + (\text{Giờ quá hạn} * \text{Giá Overtime}) - (\text{Tiền đã trả}).$ - Nếu $\text{Duration} \leq \text{Thời gian đặt trước}$: Khách không phải trả thêm 6b. Trường hợp Khách Offline (Hoặc Khách Online bị chuyển sang Offline): - Hệ thống quét tìm các khoản thanh toán "treo" (findPaymentNoShow) để bảo vệ quyền lợi khách hàng (trừ tiền đã trả nếu khách lỡ bị đẩy sang luồng Offline). $\text{Phí} = (\text{Tổng giờ} * \text{Giá Standard}) - (\text{Tiền đã trả tìm thấy}).$ (Nếu đỗ trên 24h sẽ áp dụng thêm phí overtime). 7. Hệ thống thực hiện Giao dịch (Transaction):	

- Cập nhật Spot thành Trống.
- Tạo Hóa đơn (Bill) kèm link ảnh Check-in/Check-out.
- Đóng Ticket và Reservation.

8. Hệ thống trả về thông tin Hóa đơn (Biển số, Giờ vào, Giờ ra, Số tiền khách cần trả thêm) để nhân viên thu tiền.

Ngoại lệ:

2a. AI không nhận diện được:

Trả về lỗi 400: "Không xác định được biển số/loại xe".

3a. Không tìm thấy xe trong bãi:

Hệ thống không tìm thấy Ticket active nào khớp biển số.

Trả về lỗi 404: "Xe này không tồn tại trong bãi".

7a. Lỗi tạo hóa đơn (Database Error):

Trả về lỗi 500.

Rollback toàn bộ giao dịch (Không mở cổng, không đóng ticket).

Quy tắc nghiệp vụ:

Giá quá giờ (Overtime Rate) thường cao hơn giá tiêu chuẩn để tránh việc khách chiếm chỗ quá lâu so với cam kết.

2.2.7. Quản lý nhân viên

Mô tả chức năng: Cho phép Quản trị viên (Admin) thực hiện các thao tác quản trị đối với tài khoản nhân viên bao gồm: Thêm mới, xem danh sách, chỉnh sửa thông tin, xóa tạm thời (đưa vào thùng rác) và khôi phục lại nhân viên đã xóa.

Bảng 7: Kịch bản chức năng Quản lý nhân viên

Mã Use Case	UC_07
Tên usecase	Quản lý nhân viên
Tác nhân chính	Admin
Điều kiện tiên quyết	Người dùng đăng nhập bằng tài khoản Admin
Hậu điều kiện	Cơ sở dữ liệu Staff được cập nhật tương ứng với hành động (thêm, sửa, xóa).
Chuỗi sự kiện chính: - Xem danh sách nhân viên (List Staffs): - Hệ thống hiển thị danh sách tất cả nhân viên. - Các thông tin hiển thị: tên, ngày sinh, tên tài khoản, trạng thái, thời gian tạo - Thêm mới nhân viên (Create Staff): - Admin nhập thông tin: tên, ngày sinh, tên tài khoản, mật khẩu - Hệ thống kiểm tra trùng lặp (username). - Hệ thống mã hóa mật khẩu và lưu vào DB. - Xem chi tiết và chỉnh sửa: - Admin chọn một nhân viên để xem chi tiết. - Admin cập nhật thông tin (tên, ngày sinh, mật khẩu, trạng thái) và nhấn Lưu. - Hệ thống validate và cập nhật bản ghi. - Xóa nhân viên (Soft Delete): - Admin bấm nút "Xóa" một nhân viên. - Hệ thống thực hiện Xóa mềm: Cập nhật thời gian deletedAt hiện tại cho bản ghi đó (không xóa vĩnh viễn khỏi database). - Nhân viên này biến mất khỏi danh sách chính. - Quản lý thùng rác và khôi phục:	

- Admin vào mục "Thùng rác" để xem các nhân viên đã xóa.
- Admin bấm nút "Khôi phục" (Restore).
- Hệ thống set lại deletedAt = NULL. Nhân viên quay lại danh sách hoạt động.

Ngoại lệ

2a. Trùng thông tin khi thêm mới:

- Hệ thống phát hiện tên tài khoản đã tồn tại.
- Thông báo lỗi: "Nhân viên này đã tồn tại".

Quyền truy cập bị từ chối:

- Nếu user không phải Admin gọi API.
- Middleware trả về 403 Forbidden.

Quy tắc nghiệp vụ:

Soft Delete: Hệ thống không bao giờ xóa vĩnh viễn nhân viên ngay lập tức để đảm bảo truy vết dữ liệu lịch sử (các vé xe, hóa đơn do nhân viên đó tạo trước đây).

2.2.8. Quản lý vị trí đỗ của bãi đỗ xe

Mô tả chức năng: cho phép Quản trị viên (Admin) thiết lập và cấu hình hạ tầng của bãi đỗ xe.

Bảng 8: Kịch bản chức năng Quản lý vị trí đỗ

Mã Use Case	UC_08
Tên usecase	Quản lý vị trí đỗ
Tác nhân chính	Admin
Điều kiện tiên quyết	Người dùng đăng nhập bằng tài khoản Admin
Hậu điều kiện	Cấu trúc bãi đỗ (Table Spots) thay đổi (Thêm slot, ẩn slot hoặc sửa thông tin slot)

Chuỗi sự kiện chính:

1. Xem danh sách theo khu vực (Filter by Area):

- Admin chọn một khu vực (Khu A, Khu B).
- Hệ thống trả về danh sách các Spot thuộc khu vực.

2. Xem chi tiết (View Detail):

- Admin chọn một Spot cụ thể.

- Hệ thống hiển thị: id, khu vực, vị trí, loại phương tiện, loại vé, trạng thái.

3. Thêm mới vị trí (Create Spot):

- Admin nhập: khu vực, số lượng, loại phương tiện, loại vé.

- Hệ thống kiểm tra trùng lặp (Một vị trí không thể trùng khu vực và vị trí).

- Lưu vào DB với trạng thái mặc định là Active và Status: True (Trống).

4. Chỉnh sửa vị trí (Edit Spot):

- Admin cập nhật isActive nếu tạm ngưng hoạt động để bảo trì và có thể mở để hoạt động lại.

- Hệ thống cập nhật bản ghi.

5. Xóa vị trí (Soft Delete):

- Admin chọn xóa một vị trí

- Hệ thống kiểm tra xem Spot đó có đang có xe đỗ không

- Nếu không có xe, hệ thống thực hiện Soft Delete.

6. Khôi phục vị trí (Restore):

- Admin vào thùng rác, tìm Spot đã xóa và bấm khôi phục

- Hệ thống kích hoạt lại Spot đó

Ngoại lệ

3a. Vị trí đã tồn tại:

- Admin thêm Spot "A-01" nhưng trong database đã có "A-01".

- Hệ thống báo lỗi: "Vị trí này đã tồn tại trong khu vực A".

5a. Không thể xóa vì đang có xe:

- Admin cố gắng xóa Spot đang có xe đỗ.

- Hệ thống chặn và báo lỗi: "Vị trí đang có xe đỗ, không thể xóa".

5b. Không thể xóa vì có lịch đặt trước (Reservation):

- Spot này có người đặt chỗ vào ngày mai.

- Hệ thống cảnh báo: "Vị trí này có lịch đặt chỗ sắp tới. Cần hủy lịch trước khi xóa".

Quy tắc nghiệp vụ:

- Định danh duy nhất: Một Spot được xác định bởi cặp khóa (Area + Position).

Không được phép trùng.

- Toàn vẹn dữ liệu: Khi xóa một Spot (Soft delete), các dữ liệu lịch sử (Ticket cũ, Bill cũ) gắn với Spot đó vẫn phải xem được bình thường.

- Mỗi Spot chỉ dành cho một loại xe nhất định (CAR hoặc MOTORBIKE) để phục vụ cho thuật toán tìm chỗ trống của AI.

2.2.9. Quản lý biểu phí của bãi đỗ xe

Mô tả chức năng: Cho phép Admin xem lịch sử giá vé và thiết lập mức giá mới cho từng loại xe (Ô tô/Xe máy) và từng loại vé (Tiêu chuẩn/Quá giờ). Hệ thống đảm bảo tính lịch sử bằng cách lưu trữ các mức giá cũ thay vì xóa bỏ chúng.

Bảng 9: Kịch bản chức năng Quản lý biểu phí

Mã Use Case	UC_09
Tên usecase	Quản lý biểu phí (Manage Parking Rates)
Tác nhân chính	Admin
Điều kiện tiên quyết	Người dùng đăng nhập bằng tài khoản Admin
Hậu điều kiện	- Giá vé cũ (nếu có) chuyển sang trạng thái inactive. - Một bản ghi giá vé mới được tạo với trạng thái active.
Chuỗi sự kiện chính: - Xem danh sách biểu phí (View Rates History) - Admin truy cập trang quản lý giá. - Hệ thống truy vấn toàn bộ dữ liệu từ bảng Parking Rate. - Hệ thống hiển thị danh sách bao gồm: Loại xe, Loại vé (Standard/Overtime), Đơn giá, Trạng thái (Active/Inactive), Ngày áp dụng (createdAt). - Thiết lập giá mới (Set New Rate): - Admin chọn: Loại xe (vehicle Type), Loại vé (ticket Type) và nhập Đơn giá mới (unit Price). - Admin nhấn "Lưu/Cập nhật". - Hệ thống thực hiện Transaction (Giao dịch):	

+ Bước 1: Tìm tất cả các bản ghi cùng loại xe, loại vé đang active và chuyển sang inactive. + Bước 2: Tạo bản ghi mới với giá vừa nhập và đặt ở trạng thái active. - Hệ thống thông báo thành công.
Ngoại lệ: 2a. Dữ liệu đầu vào thiếu: - Admin bỏ trống đơn giá hoặc loại xe. - Hệ thống trả về lỗi 404: "Vui lòng gửi đầy đủ dữ liệu đầu vào". 2b. Lỗi Server/Database: - Quá trình Transaction bị lỗi ở bước 2. - Hệ thống Rollback (hoàn tác) lại bước 1 (khôi phục trạng thái active cho giá cũ). - Thông báo lỗi 500: "Lỗi server vui lòng thử lại sau".
Quy tắc nghiệp vụ: - Nguyên tắc "Một giá duy nhất": Tại một thời điểm, với một cặp (Loại xe + Loại vé) chỉ được phép có duy nhất 1 bản ghi có trạng thái active. - Không xóa dữ liệu (No Hard Delete): không xóa hoặc ghi đè giá cũ. Việc giữ lại giá cũ (inactive) giúp hệ thống tính toán lại doanh thu của các hóa đơn trong quá khứ một cách chính xác.

2.2.10 Xem báo cáo doanh thu theo các tháng

Mô tả chức năng: Chức năng cho phép Admin xem biểu đồ doanh thu tổng hợp của 12 tháng trong năm hiện tại

Bảng 10: Kịch bản chức năng xem báo cáo doanh thu theo tháng

Mã Usecase	UC_10
Tên usecase	Xem báo cáo doanh thu theo tháng
Tác nhân chính	Admin
Điều kiện tiên quyết	Người dùng đăng nhập bằng tài khoản Admin
Hậu điều kiện	Admin xem được doanh thu của năm nay
Chuỗi sự kiện chính: 1. Admin truy cập vào trang Dashboard 2. Frontend gọi API GET /revenue/monthly	

3. Hệ thống xác định năm hiện tại (Current Year)
4. Truy vấn cơ sở dữ liệu (Aggregation):
- Hệ thống lọc các hóa đơn có finish Time trong năm nay.
- Hệ thống tính tổng doanh thu: SUM (payed Money + total Price).
- Gom nhóm dữ liệu theo tháng (GROUP BY MONTH).
5. Xử lý logic chuẩn hóa dữ liệu (Data Normalization):
- Hệ thống tạo một vòng lặp từ Tháng 1 đến Tháng 12.
- Với mỗi tháng, hệ thống kiểm tra xem CSDL có trả về dữ liệu không.
Nếu CÓ: Lấy giá trị tổng tiền từ database.
Nếu KHÔNG: Gán giá trị doanh thu = 0.
6. Hệ thống trả về kết quả JSON gồm 2 mảng: labels (Tên tháng) và data (Doanh thu tương ứng).
7. Frontend nhận dữ liệu và vẽ biểu đồ (Bar Chart).

2.2.11 Xem lưu lượng xe vào ra theo các khung giờ trong một ngày

Mô tả chức năng: Cho phép Admin xem biểu đồ mật độ xe vào và ra trong một ngày cụ thể, được phân chia chi tiết theo 24 khung giờ (từ 0h đến 23h).

Bảng 11: Kịch bản chức năng Thống kê lưu lượng xe

Mã usecase	UC_11
Tên usecase	Thống kê lưu lượng xe (Traffic Flow Statistics)
Tác nhân chính	Admin
Điều kiện tiên quyết	Người dùng đăng nhập bằng tài khoản Admin và chọn traffic-flow
Hậu điều kiện	Người dùng xem được được lưu lượng xe theo từng khung giờ trong ngày mà người dùng chọn.
Chuỗi sự kiện chính: - Admin chọn ngày cần xem báo cáo trên giao diện (DatePicker). - Frontend gửi request lên hệ thống kèm theo date. - Hệ thống khởi tạo 2 mảng rỗng (kích thước 24) để chứa dữ liệu đếm (Bucket Array). - Hệ thống truy vấn cơ sở dữ liệu lấy tất cả các vé xe được tạo trong ngày đó.	

5. Hệ thống xử lý lặp qua từng vé và trả về kết quả JSON.

6. Frontend vẽ biểu đồ đường (bar chart)

Ngoại lệ

2a. Không chọn ngày:

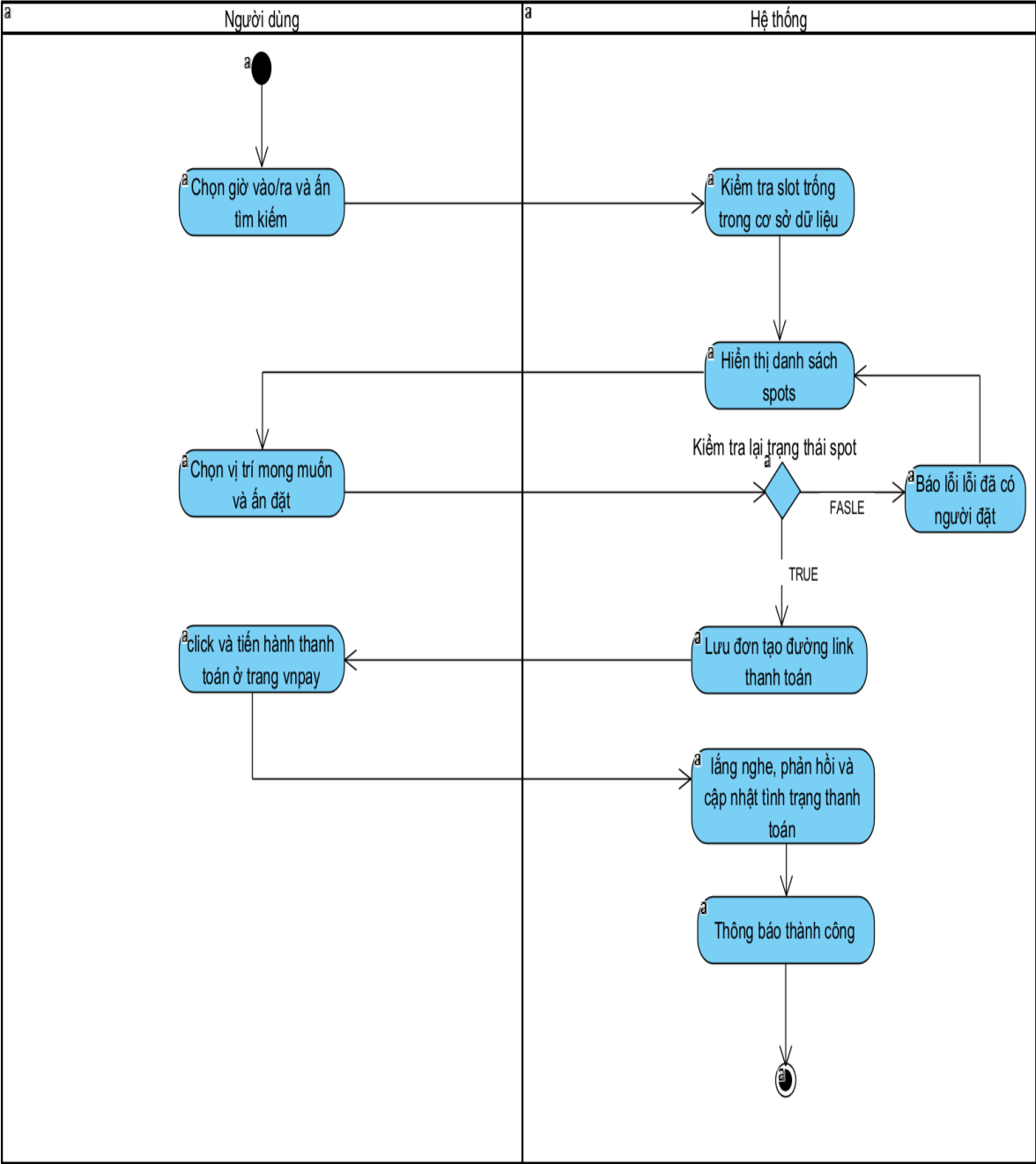
- Người dùng gửi body thiếu date.
- Hệ thống báo lỗi 404: "Vui lòng gửi ngày muốn thống kê".

4a. Không có dữ liệu:

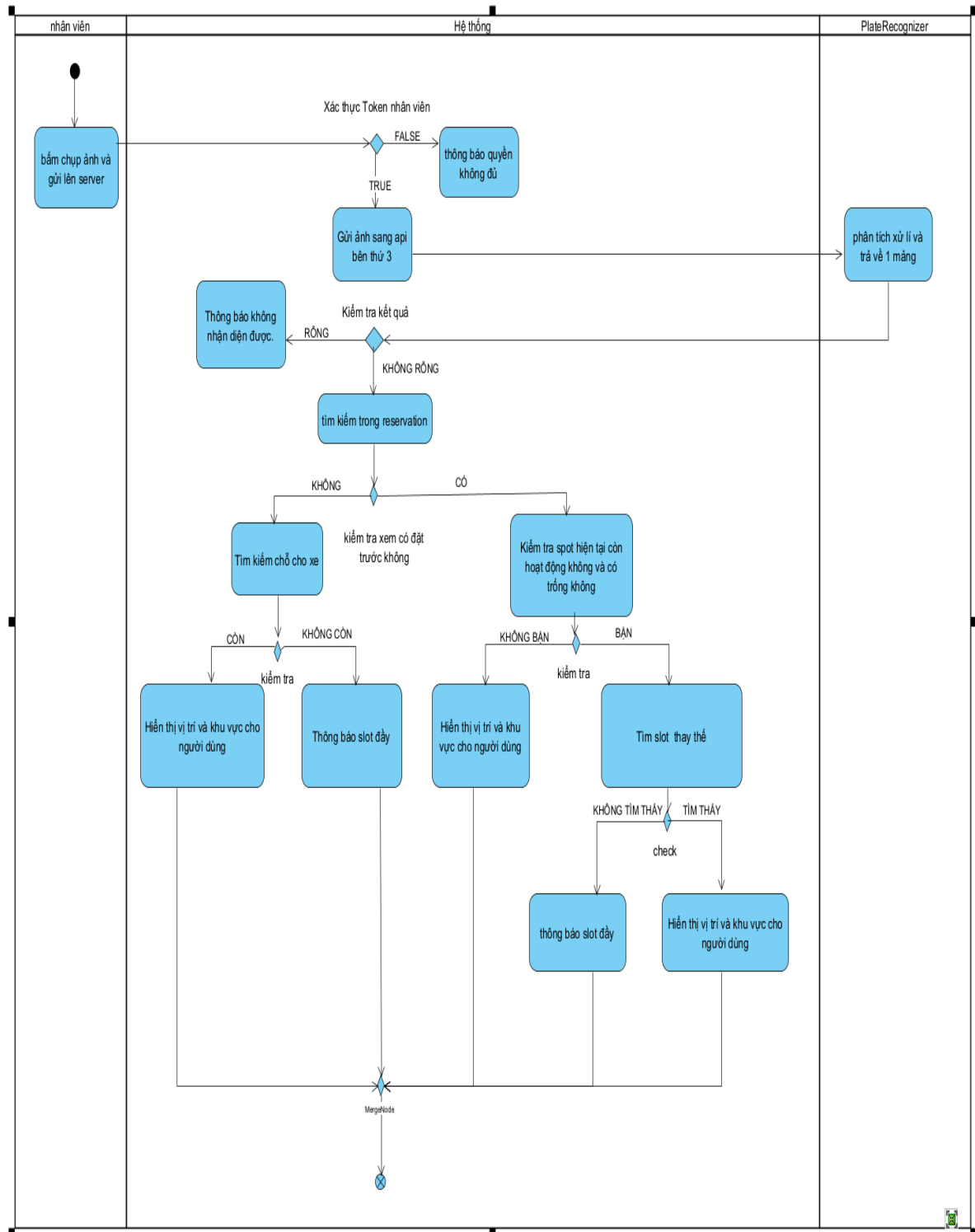
- Ngày đó bãi xe đóng cửa hoặc không có khách.
- Hệ thống trả về 2 mảng toàn số 0.

2.3. Biểu đồ hoạt động

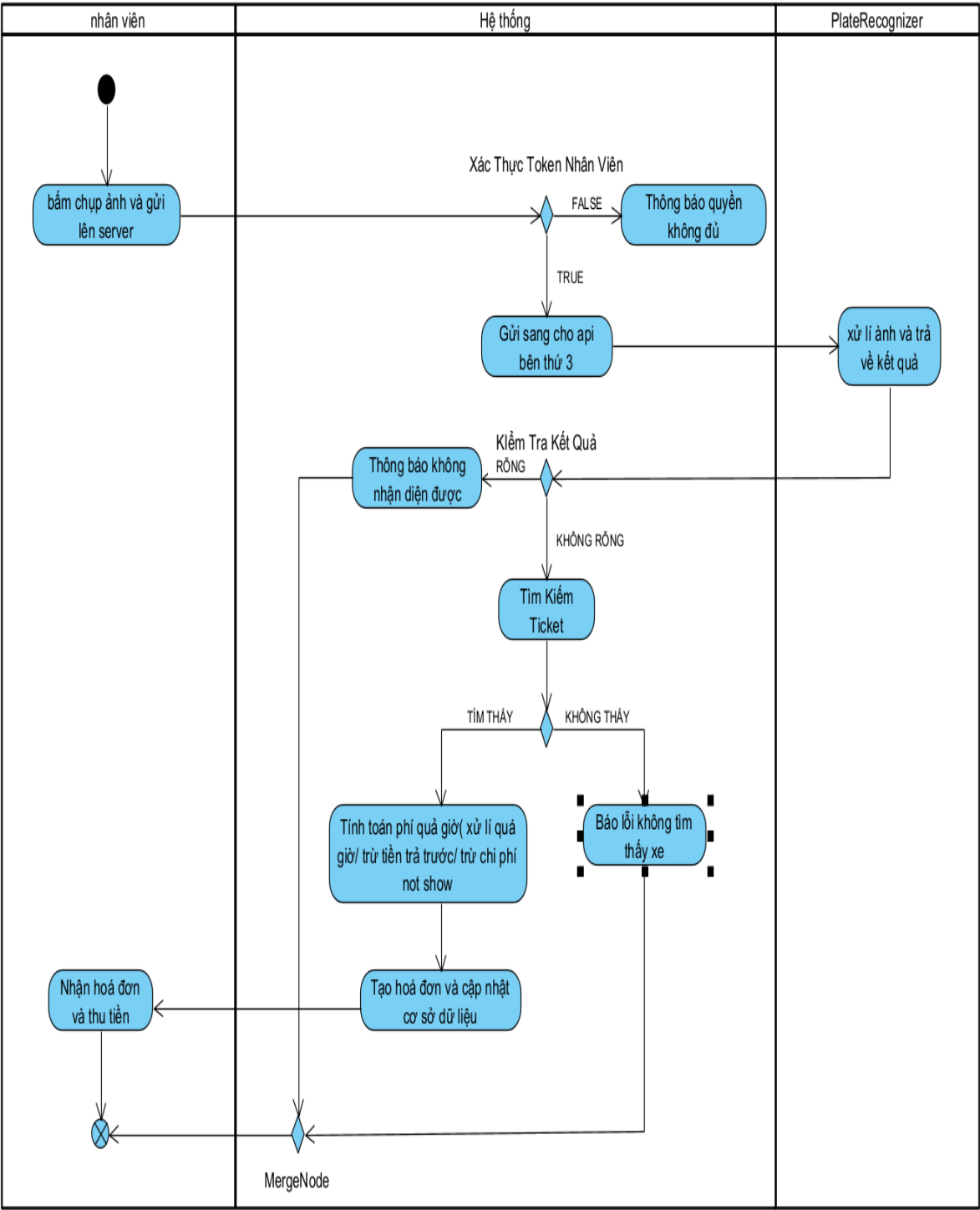
2.3.1 Module đặt chỗ



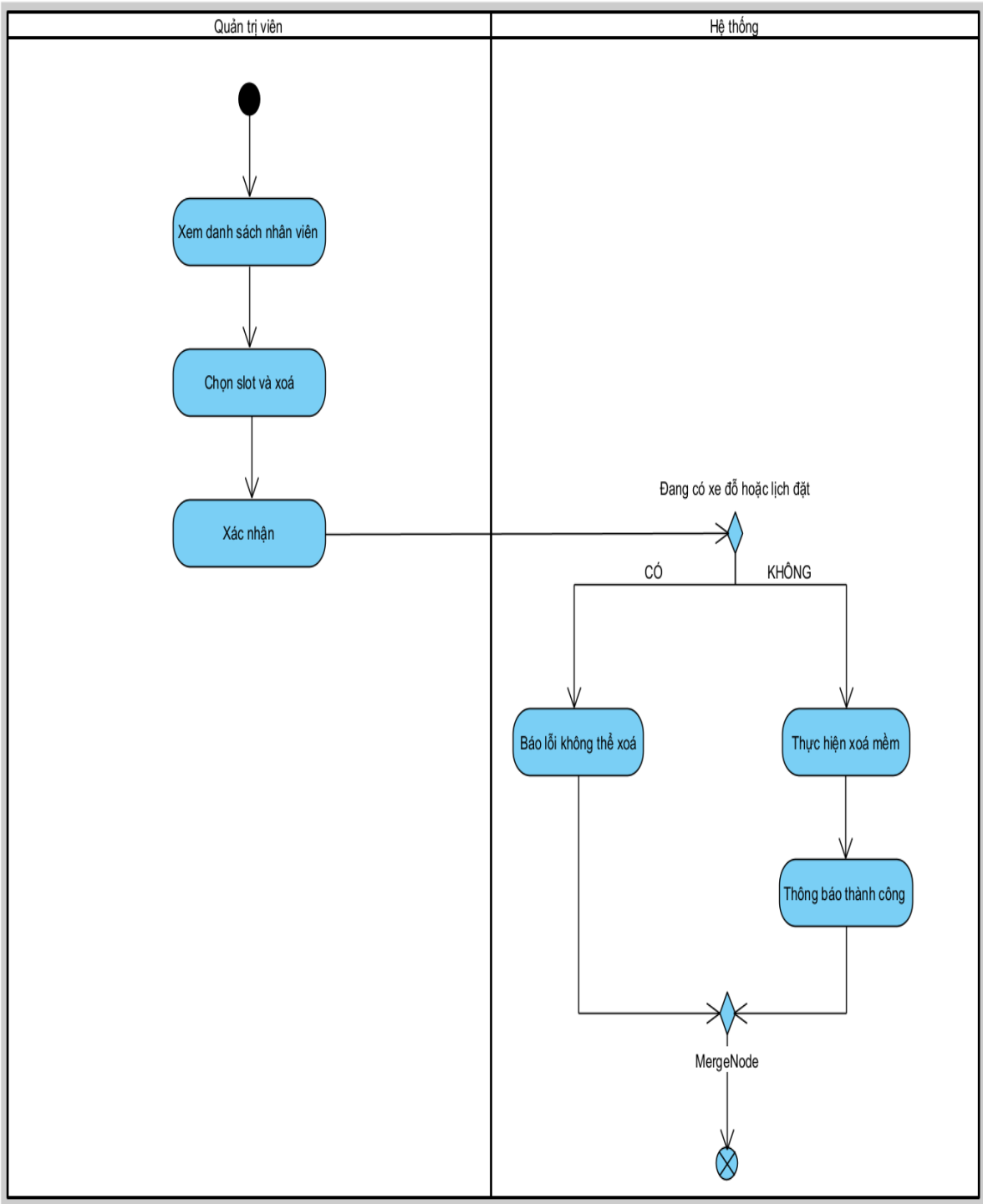
2.3.2. Module check in khi xe vào



2.3.3. Module check out khi xe ra

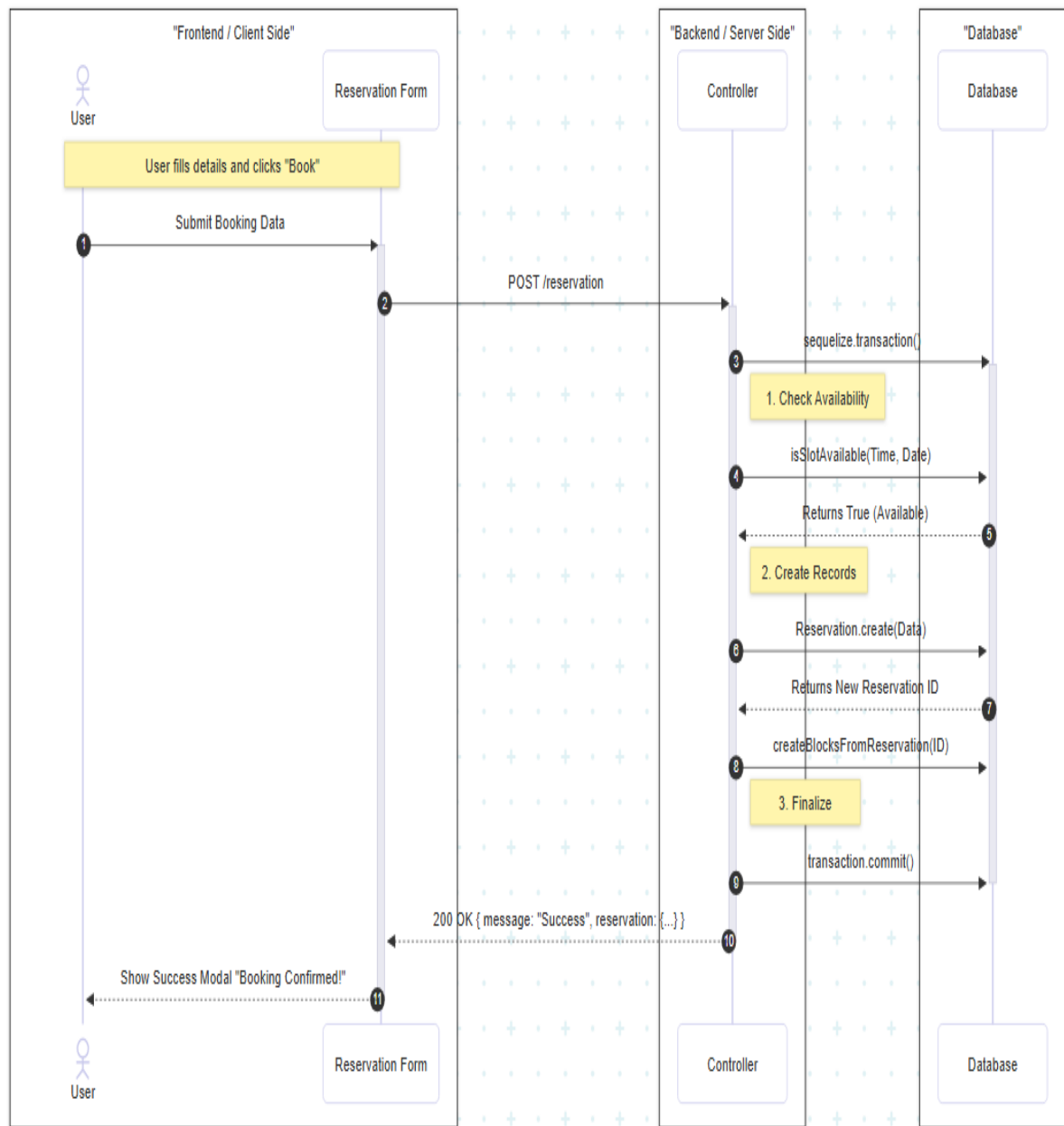


2.3.4. Module xoá spot

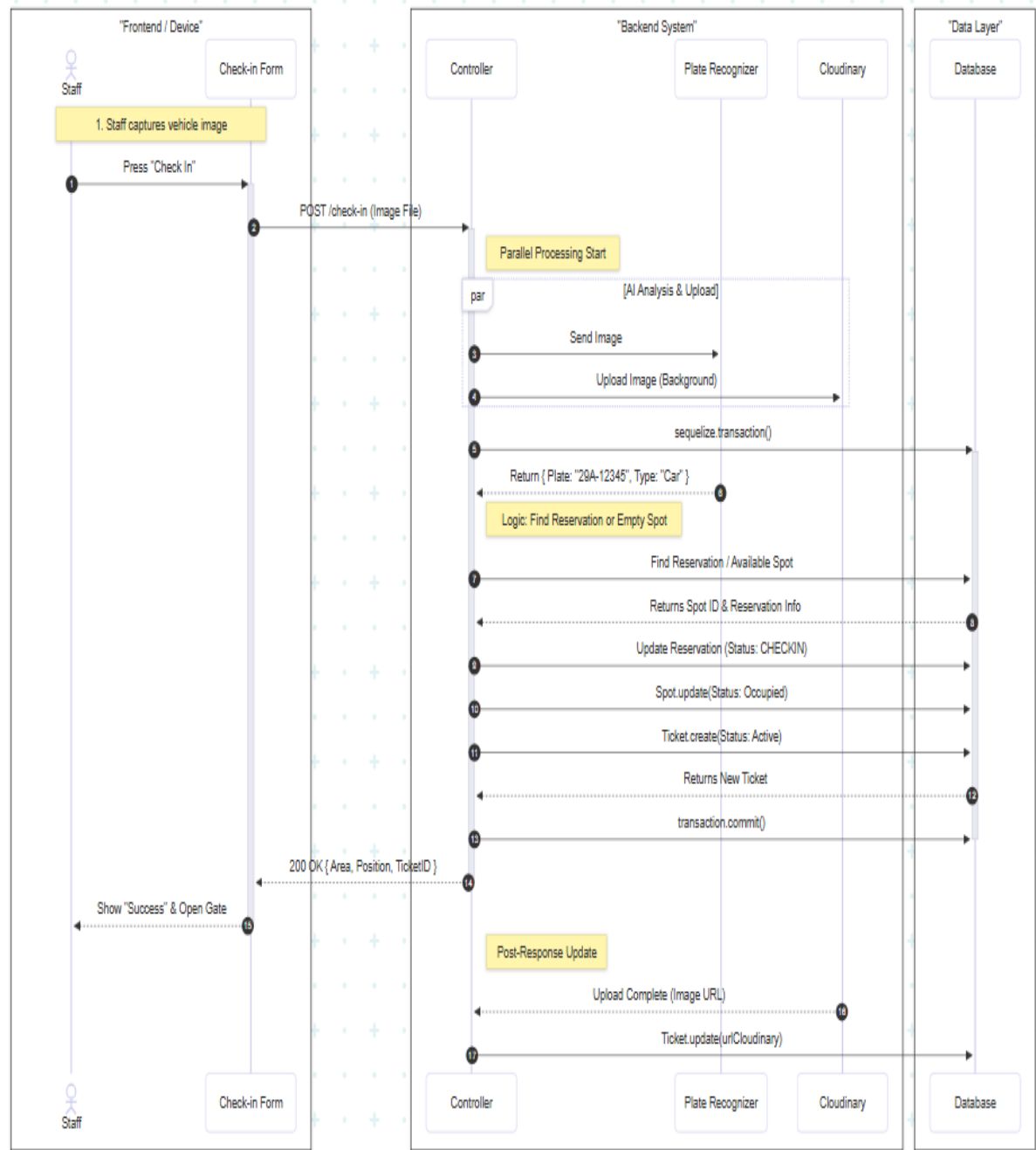


2.4. Biểu đồ tuần tự

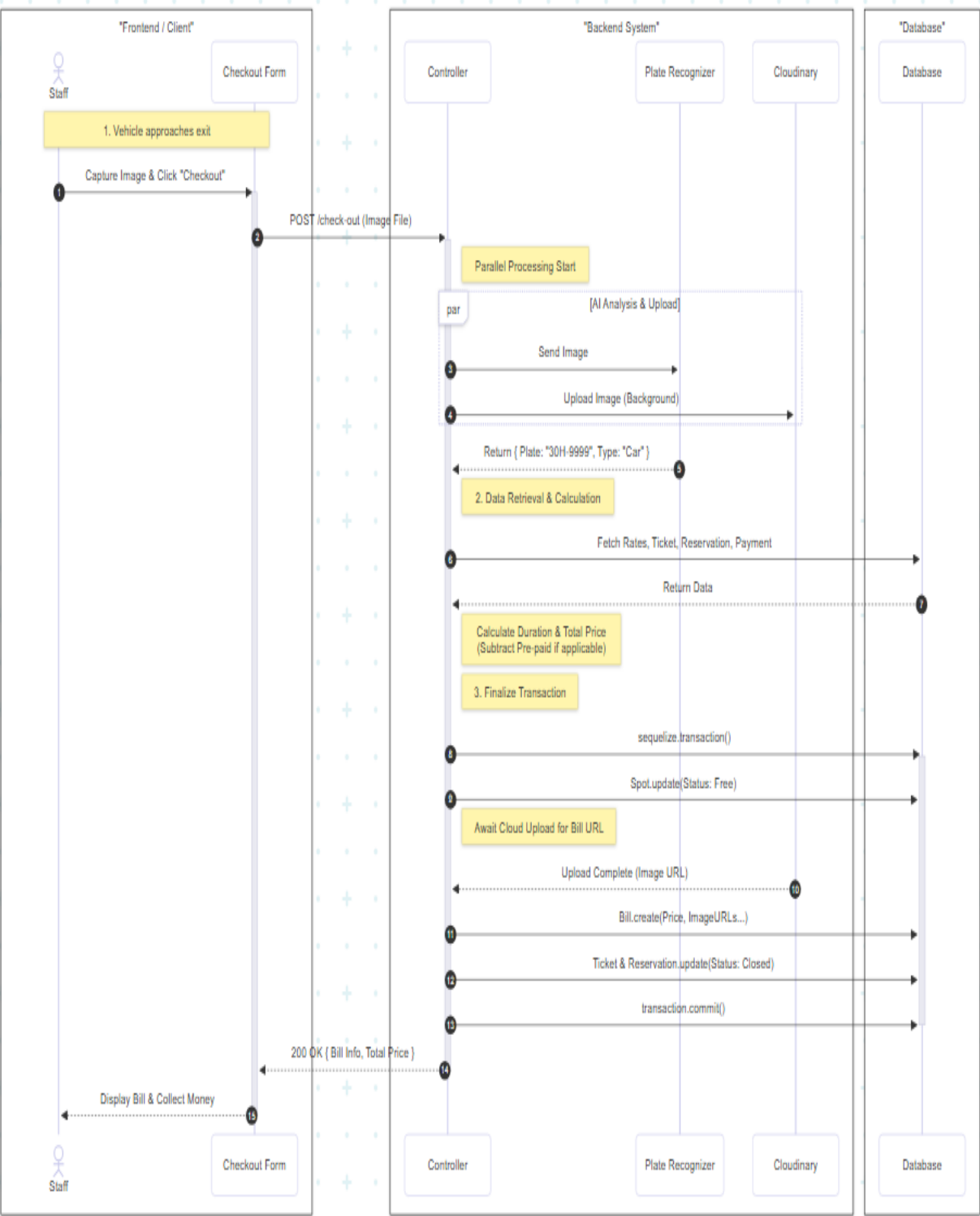
2.4.1. Module đặt chỗ



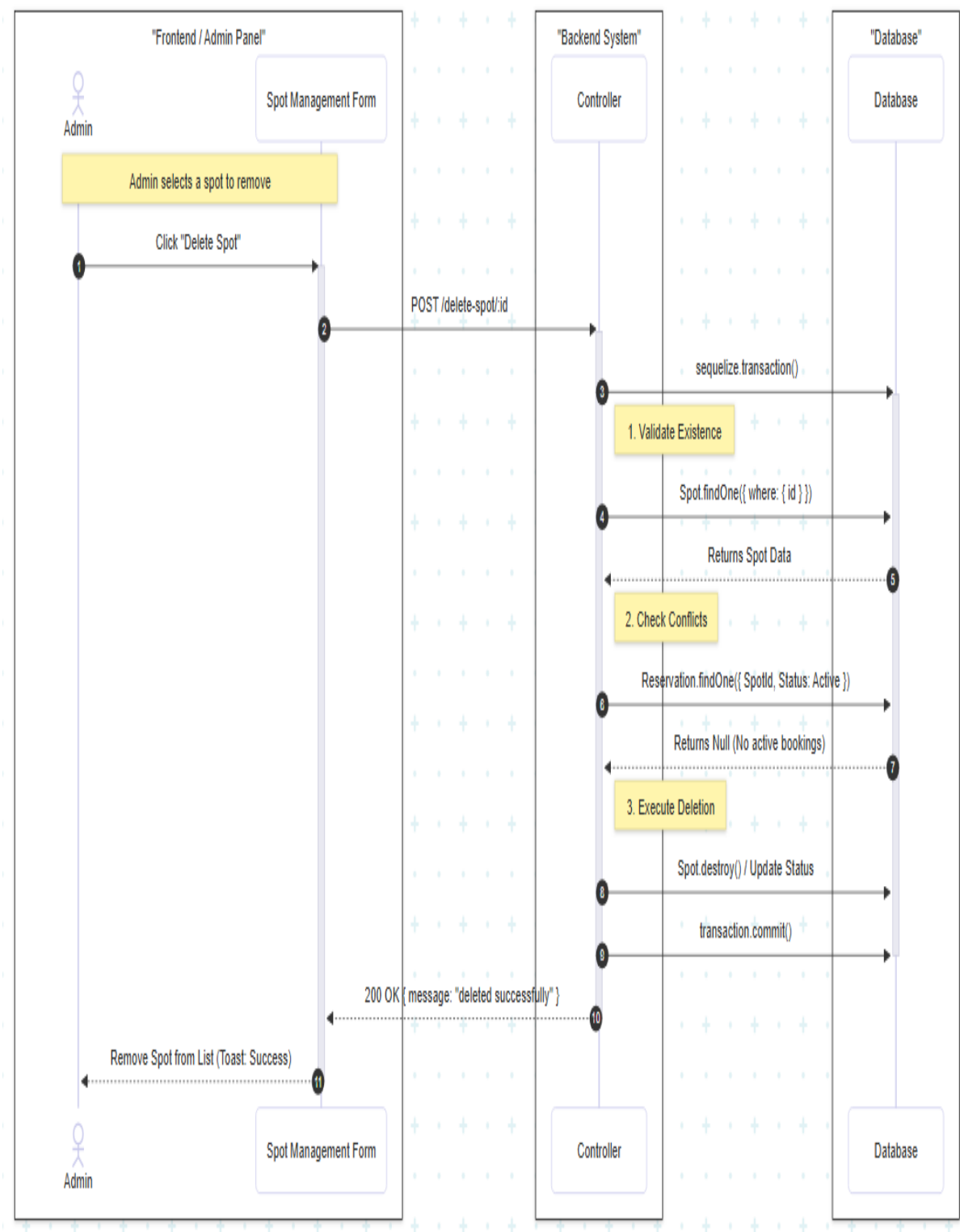
2.4.2. Module check in



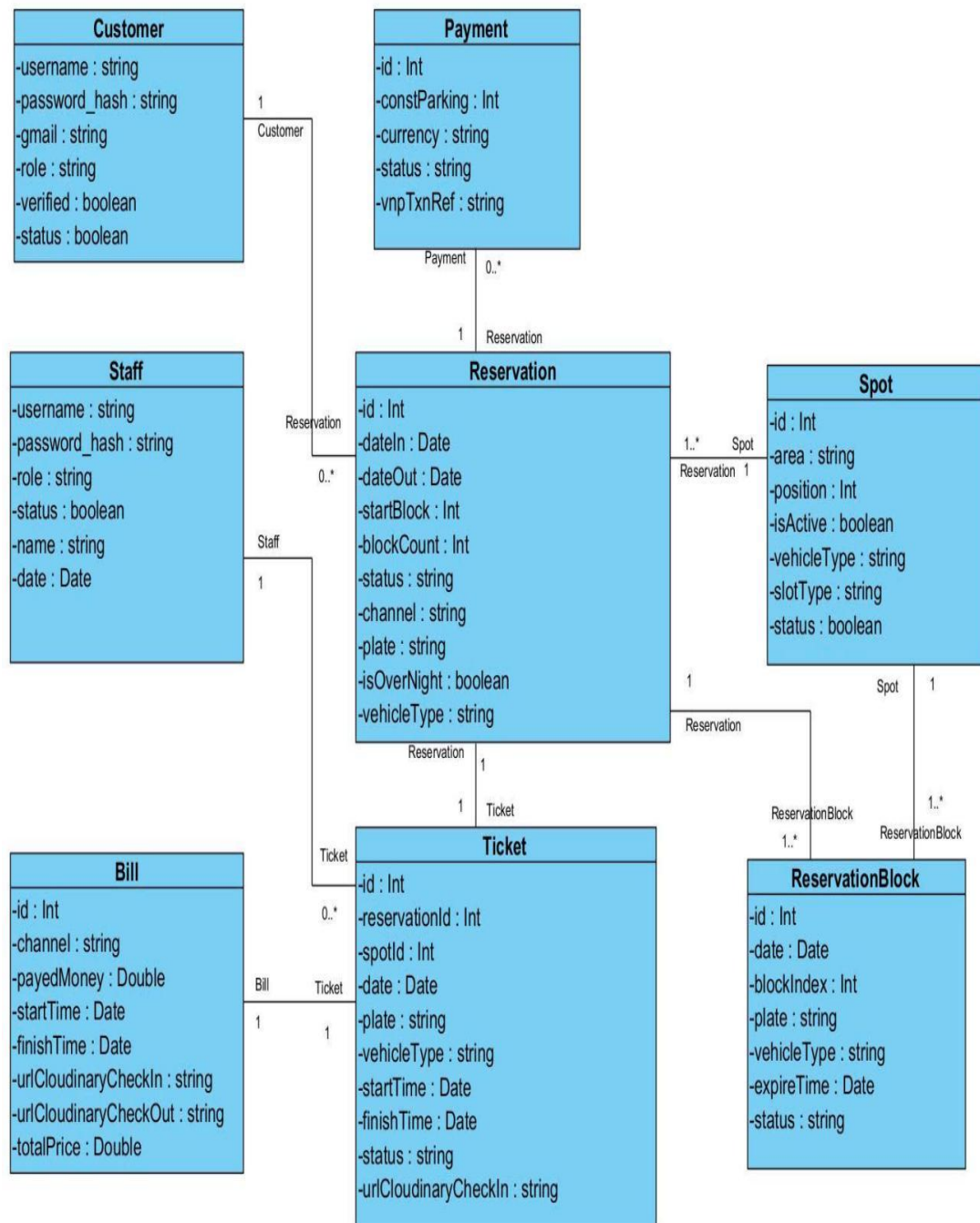
2.4.3. Module check out



2.4.4. Module xóa spot



2.5. Biểu đồ lớp



Hình 1: Biểu đồ lớp

Giải thích biểu đồ lớp:

2.5.1. Nhóm Người dùng & Quản trị

2.5.1.1. Customer (Khách hàng)

Đại diện cho người dùng cuối sử dụng ứng dụng để đặt chỗ.

username (String)	Số điện thoại
password_hash (String):	Mật khẩu đã mã hóa
gmail (String)	Địa chỉ email liên lạc/xác thực.
role (String)	Vai trò
verified (boolean)	Trạng thái xác thực email (True/False)
status (boolean)	Trạng thái hoạt động của tài khoản (Khóa/Mở).

2.5.1.2. Staff (Nhân viên)

Đại diện cho nhân viên quản lý hoặc bảo vệ bãi xe.

username(String)	Tên đăng nhập
password_hash (String)	Mật khẩu đã mã hóa.
name(String)	Tên người dùng
Date (Date)	Ngày sinh
role (String)	Phân biệt admin và staff
status (boolean)	Trạng thái hoạt động

2.5.2. Nhóm Quản lý Bãi đỗ

2.5.2.1. Spot (Vị trí đỗ xe)

Đại diện cho một ô đỗ xe vật lý trong bãi.

id (Int)	Mã định danh vị trí.
area (String)	Khu vực đỗ
position (Int)	Số thứ tự vị trí trong khu vực.
isActive (boolean)	Cờ đánh dấu vị trí có đang bảo trì hay không.
slotType (String)	Loại slot (online hoặc offline)

status (boolean)	Trạng thái hiện tại (Trống/Đầy)
------------------	---------------------------------

2.5.2.2. *Reservation (Đơn đặt chỗ)*

Lưu trữ thông tin khi khách hàng đặt chỗ trước qua ứng dụng.

id (Int)	Mã đơn đặt
dateIn (Date)	Ngày dự kiến vào.
dateOut (Date)	Ngày dự kiến ra.
startBlock (Int)	Index của khung giờ bắt đầu
blockCount (Int)	Tổng số block thời gian đã đặt
status (String)	Trạng thái đặt chỗ
channel (String)	Kênh đặt(online hay offline)
plate (String)	Biển số xe đăng ký
vehicleType	Loại xe
isOverNight	Cờ đánh dấu có gửi qua đêm hay không

2.5.2.3. *Reservation Block* (Chi tiết khung giờ đặt)

Bảng trung gian để quản lý lịch trống/bận của từng Spot theo khung giờ nhỏ.

id (Int)	Mã định danh.
date (Date)	Ngày cụ thể
blockIndex (Int)	Số thứ tự khung giờ trong ngày(0-23)
plate (String)	Biển số xe chiếm giữ block này.
vehicleType (String)	Loại xe
expireTime (Date)	Thời gian hết hạn để thanh toán
status (String)	Trạng thái block

2.5.3. Nhóm Vận hành & Thanh toán

2.5.3.1. *Ticket* (Vé xe/Lượt gửi thực tế)

Được sinh ra khi xe thực sự Check-in vào bãi.

id (Int)	Mã vé
----------	-------

reservationId(Int)	Mã đơn đặt chỗ
date(Date)	Ngày tạo ticket
plate(String)	Biển số xe
vehiclyType(String)	Loại xe
startTime (Date)	Thời gian thực tế xe vào (Check-in).
finishTime (Date)	Thời gian thực tế xe ra (Check-out)
status (String)	Trạng thái vé
urlCloudinaryCheckIn (String)	Link ảnh chụp biển số lúc vào

2.5.3.2. Bill (Hóa đơn)

Thông tin thanh toán cuối cùng khi xe Check-out.

id (Int)	Mã hóa đơn.
channel (String)	Phân biệt khách đến offline hay online
payedMoney (Double)	Số tiền đã trả(cho khách đặt trước)
startTime (Date)	Thời gian bắt đầu tính tiền
finishTime (Date)	Thời gian kết thúc tính tiền.
urlCloudinaryCheckIn (String)	Ảnh bằng chứng lúc vào.
urlCloudinaryCheckOut (String)	Ảnh bằng chứng lúc ra
totalPriece (Double)	Số tiền cần phải trả

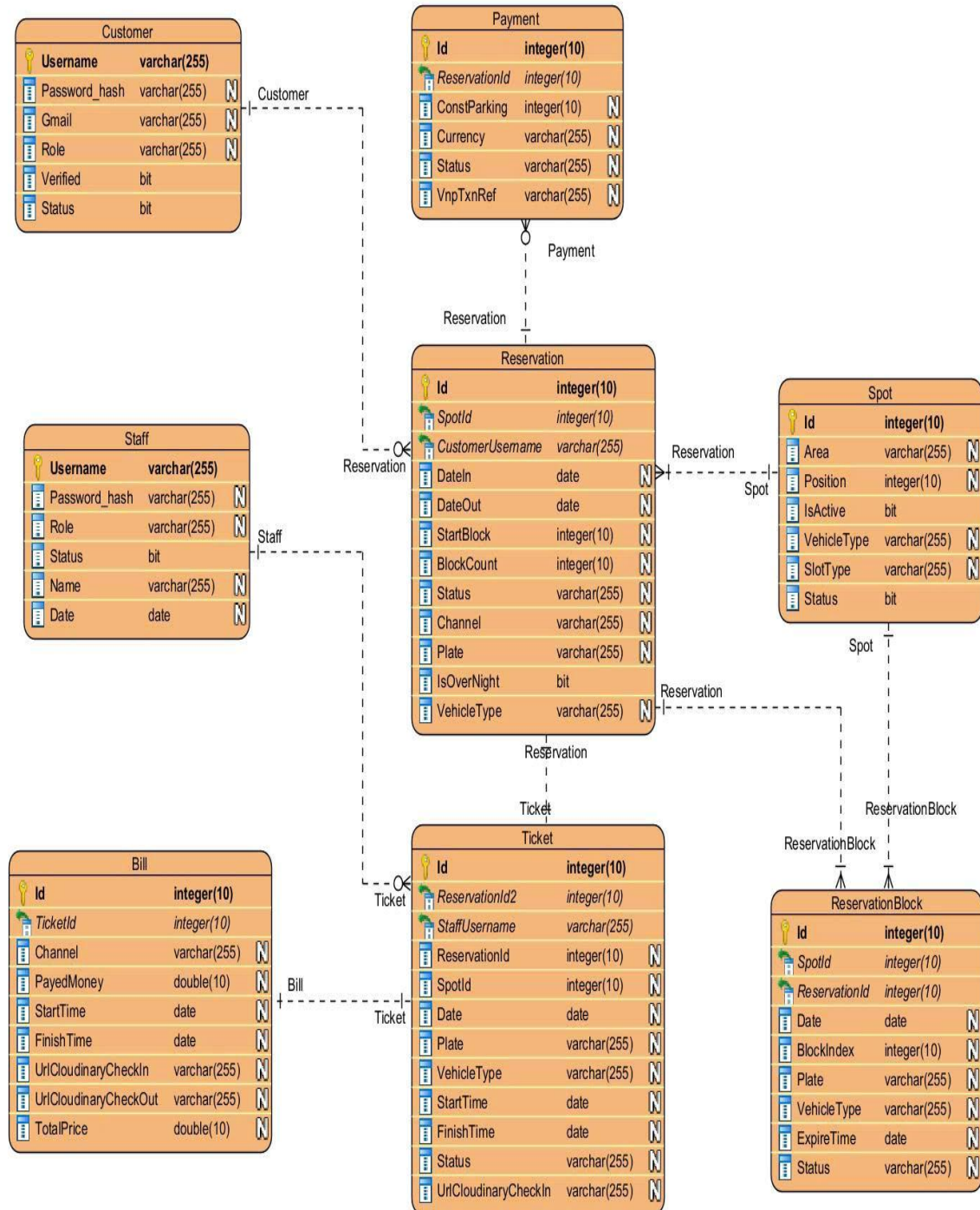
2.5.3.3. Payment (Giao dịch thanh toán)

Lịch sử thanh toán qua VNPay.

id (Int)	Mã giao dịch.
constParking (Int)	Số tiền thanh toán
currency (String)	Đơn vị tiền tệ (VND)
status (String)	Trạng thái giao dịch
vnpTxnRef (String)	Mã tham chiếu giao dịch từ cổng VNPay.

2.6. Lược đồ cơ sở dữ liệu

Mô tả lược đồ cơ sở dữ liệu:



Hình 2: Cơ sở dữ liệu

2.6.1. Bảng customer (Khách hàng)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Username (PK)	varchar(255)	Tên đăng nhập duy nhất của khách hàng (Khóa chính).
Password_hash	varchar(255)	Mật khẩu đã được mã hóa (dùng bcrypt) để bảo mật.
Gmail	varchar(255)	Địa chỉ email dùng để nhận mã OTP và thông báo.
Role	varchar(255)	Vai trò trong hệ thống (Mặc định là 'CUSTOMER').
Verified	Boolean	Trạng thái xác thực email (1: Đã xác thực, 0: Chưa).
Status	Boolean	Trạng thái tài khoản (1: Hoạt động, 0: Bị khóa).

2.6.2 Bảng staff (Nhân viên)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Username (PK)	varchar(255)	Tên đăng nhập của nhân viên (Khóa chính).
Password_hash	varchar(255)	Mật khẩu đăng nhập đã mã hóa.
Name	varchar(255)	Tên của người dùng
date	date	Ngày sinh của người dùng
Role	varchar(255)	Vai trò
Status	bit (Boolean)	Trạng thái hoạt động của nhân viên.

2.6.3. Bảng spot (Vị trí đỗ)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Id (PK)	integer(10)	Mã định danh duy nhất của vị trí đỗ.
Area	varchar(255)	Khu vực đỗ
Position	integer(10)	Số thứ tự vị trí trong khu vực
IsActive	bit	Đánh dấu vị trí có đang bảo trì hay không (1: Dùng được, 0: Bảo trì).

VehicleType	varchar(255)	Loại xe được phép đỗ ('CAR' hoặc 'MOTORBIKE') - Dùng để AI so khớp.
SlotType	varchar(255)	Loại ô đỗ
Status	bit	Trạng thái hiện tại (1: Trống, 0: Đang có xe đỗ).

2.6.4. Bảng reservation (Đơn đặt chỗ)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Id (PK)	integer(10)	Mã đơn đặt chỗ.
SpotId (FK)	integer(10)	Khóa ngoại liên kết với bảng Spot
CustomerUsername (FK)	varchar(255)	Khóa ngoại liên kết với bảng Customer
DateIn	date	Ngày giờ dự kiến vào.
DateOut	date	Ngày giờ dự kiến ra.
StartBlock	integer(10)	Chỉ số block thời gian bắt đầu (dùng cho thuật toán chia nhỏ giờ).
BlockCount	integer(10)	Tổng số block thời gian đã đặt.
Status	varchar(255)	Trạng thái đơn ('PENDING', 'BOOKED', 'CANCELLED', 'CHECKOUT').
Channel	varchar(255)	Kênh đặt (online/offline).
Plate	varchar(255)	Biển số xe đăng ký đặt chỗ.
VehicleType	varchar(255)	Loại xe
IsOverNight	boolean	Đánh dấu xe có gửi qua đêm hay không.

2.6.5. Bảng reservation block (Khối thời gian giữ chỗ)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Id (PK)	integer(10)	Mã định danh khối thời gian.
SpotId (FK)	integer(10)	Liên kết với vị trí đỗ.
ReservationId (FK)	integer(10)	Liên kết với đơn đặt chỗ gốc.
Date	date	Ngày của block này.

BlockIndex	integer(10)	Chỉ số thời gian
Plate	varchar(255)	Biển số xe
VehicleType	varchar(255)	Loại xe.
ExpireTime	date	Thời gian hết hạn giữ chỗ (dùng cho Cron Job hủy đơn Pending).
Status	varchar(255)	Trạng thái

2.6.6. Bảng ticket (Vé xe / Phiên gửi xe)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Id (PK)	integer(10)	Mã vé xe.
ReservationId (FK)	integer(10)	Liên kết với đơn đặt chỗ
StaffUsername (FK)	varchar(255)	Nhân viên giám sát
SpotId (FK)	integer(10)	Vị trí xe đang đỗ thực tế.
Date	date	Ngày tạo vé.
Plate	varchar(255)	Biển số xe nhận diện được lúc vào.
VehicleType	varchar(255)	Loại xe nhận diện được.
StartTime	date	Thời gian check-in thực tế.
FinishTime	date	Thời gian check-out thực tế (Cập nhật khi xe ra).
Status	varchar(255)	Trạng thái vé
UrlCloudinaryCheckIn	varchar(255)	Đường dẫn ảnh chụp biển số lúc vào (Lưu trên Cloudinary).

2.6.7. Bảng payment (Giao dịch thanh toán Online)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Id (PK)	integer(10)	Mã giao dịch.
ReservationId (FK)	integer(10)	Liên kết với đơn đặt chỗ được thanh toán.
ConstParking	integer(10)	Số tiền đã thanh toán.
Currency	varchar(255)	Đơn vị tiền tệ ('VND').

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Status	varchar(255)	Trạng thái giao dịch từ VNPay
VnpTxnRef	varchar(255)	Mã tham chiếu giao dịch của VNPay.

2.6.8. Bảng bill (Hóa đơn cuối cùng)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
Id (PK)	integer(10)	Mã hóa đơn.
TicketId (FK)	integer(10)	Liên kết với vé xe.
Channel	varchar(255)	Kênh thanh toán cuối cùng(Tiền mặt/Chuyển khoản).
PayedMoney	double(10)	Số tiền khách đã trả trước đó(từ bảng Payment).
StartTime	date	Thời gian bắt đầu tính tiền.
FinishTime	date	Thời gian kết thúc tính tiền.
TotalPrice	integer(10)	Tổng số tiền khách phải trả cuối cùng (sau khi trừ cọc/cộng phạt).
UrlCloudinaryCheckIn	varchar(255)	Ảnh bằng chứng lúc vào
UrlCloudinaryCheckOut	varchar(255)	Ảnh bằng chứng lúc ra.

CHƯƠNG 3: CÀI ĐẶT VÀ THỰC NGHIỆM

3.1. Môi trường cài đặt và Cấu trúc dự án

3.1.1. Yêu cầu phần cứng và phần mềm:

Hệ thống được xây dựng và thử nghiệm trên môi trường phát triển (Localhost) với cấu hình cụ thể như sau:

Cấu hình phần cứng:

CPU: Intel Core i5 (hoặc tương đương) trở lên.

RAM: Tối thiểu 8GB (Khuyến dùng 16GB để chạy đồng thời Server, Client và Database).

Ổ cứng: SSD (để đảm bảo tốc độ đọc ghi dữ liệu nhanh).

Kết nối mạng: Ổn định (Bắt buộc để kết nối các API bên thứ 3 như PlateRecognizer, Cloudinary, VNPay).

Môi trường phần mềm:

Thành phần	Công nghệ/Phiên bản	Vai trò
Hệ điều hành	Windows 10/11 (64-bit)	Môi trường chạy hệ thống
Runtime	Node.js v18.x (LTS)	Môi trường thực thi JavaScript server-side
Database	MySQL workbench 8.0	Hệ quản trị cơ sở dữ liệu quan hệ
Frontend	Next.js 13/14	Framework xây dựng giao diện người dùng
Công cụ quản lý	Git & GitHub	Quản lý phiên bản mã nguồn
IDE	Visual Studio Code	Trình soạn thảo mã nguồn

3.1.2. Cấu trúc tổ chức dự án (Project Structure)

Hệ thống được chia thành hai thư mục chính độc lập: Backend (API Server) và Frontend (Client App)

Cấu trúc Backend (Node.js + Express + Sequelize): Tuân thủ mô hình MVC (Model-View-Controller):

config: Chứa file cấu hình database, vnpay, cloudinary, mailer

models: Định nghĩa các thực thể

controllers: Chứa logic xử lý nghiệp vụ.

routes: Định nghĩa các API Endpoint.

middlewares: Xử lý xác thực (JWT Auth).

Cấu trúc Frontend (Next.js + TypeScript):

App: Là thư mục trung tâm quản lý routing của ứng dụng theo cơ chế file-based routing của Next.js. Thư mục này chứa toàn bộ các trang giao diện cho từng nhóm người dùng như Admin, Customer và Staff, đồng thời sử dụng layout.tsx để định nghĩa bố cục chung cho từng khu vực chức năng.

Components chứa các component giao diện tái sử dụng trong toàn bộ hệ thống. Các component này bảo vệ route theo vai trò (Admin, Customer, Staff) và có các component giao diện dùng chung như Sidebar, bảng biểu, biểu đồ thống kê.

Thư mục UI chứa các component UI cơ bản (Button, Table, Dialog, Form, Toast) được xây dựng theo hướng chuẩn hóa, giúp đảm bảo tính nhất quán về giao diện và trải nghiệm người dùng.

Contexts chứa các Context API dùng để quản lý trạng thái toàn cục liên quan đến xác thực và thông tin người dùng (Customer, Staff). Việc này giúp chia sẻ trạng thái đăng nhập và thông tin người dùng xuyên suốt ứng dụng mà không cần truyền props phức tạp.

Hooks chứa các custom hooks, phục vụ cho việc tái sử dụng logic xử lý phía Client, ví dụ như hiển thị thông báo (toast), giúp code gọn gàng và dễ bảo trì hơn.

Lib Là tầng logic dùng chung và giao tiếp Backend, bao gồm: các tiện ích xử lý token, socket realtime và hàm hỗ trợ chung. Thư mục api trong Lib đóng vai trò tương đương với service layer, nơi định nghĩa các hàm gọi API xuống Backend thông qua HTTP/HTTPS (Axios client), được phân chia rõ ràng theo từng vai trò Admin, Customer và Staff. Việc tách riêng tầng API giúp frontend dễ dàng thay đổi hoặc mở rộng Backend mà không ảnh hưởng trực tiếp đến giao diện.

3.1.3. Triển khai các tính năng của hệ thống

Chức năng dành cho người dùng

Mã	Tên tính năng	Trạng thái	Ghi chú
USER_01	Đăng ký và Xác thực	Hoàn thành	Đã hoàn thành luồng đăng ký, gửi Email xác thực (Link) và kích hoạt tài khoản.
USER_02	Đăng nhập và Bảo mật	Hoàn thành	Đã hoàn thành đăng nhập phân quyền, sử dụng JWT để bảo mật phiên làm việc.
USER_03	Tìm kiếm vị trí đỗ và đặt chỗ	Hoàn thành	Đã hoàn thành việc hiển thị danh sách Spot theo khung giờ tìm kiếm và đặt chỗ thành công
USER_04	Thanh toán điện tử	Hoàn thành	Đã tích hợp thành công cổng VNPay (Sandbox)
USER_05	Xem lịch sử đặt chỗ	Hoàn thành	Đã hoàn thành việc hiển thị reservation của khách hàng
USER_06	Xem số lượng chỗ hiện tại còn trống	Hoàn thành	Người dùng không cần đăng nhập mà vẫn xem được số lượng spots thực tế đang khả thi, giúp tiết kiệm cho khách hàng tìm kiếm bãi đỗ xe
USER_07	Xác minh lại tài khoản	Hoàn thành	Người dùng xác minh lại tài khoản khi chưa xác minh khi đăng ký thành công do xoá mất email gửi xác minh

Chức năng dành cho Quản trị viên và Nhân viên

Mã	Tên tính năng	Trạng thái	Ghi chú
ADMIN_01	Quản lý hạ tầng (Spots)	Hoàn thành	Đã hoàn thành chức năng Thêm/Sửa/Xóa vị trí đỗ. Có kiểm tra ràng buộc dữ liệu (không xóa khi đang có xe).
ADMIN_02	Quản lý bảng giá	Hoàn thành	Đã hoàn thành việc cấu hình giá vé theo khung giờ và loại xe (Ô tô/Xe máy).

ADMIN_03	Quản lý nhân sự	Hoàn thành	Đã hoàn thành luồng tạo tài khoản cho nhân viên.
ADMIN_04	Thống kê doanh thu	Hoàn thành cơ bản	Đã hoàn thành việc tính tổng doanh thu theo tháng và luồng phương tiện vào ra các khu giờ trong một ngày. Bên cạnh đó, quản trị viên có thể xem doanh thu thực tế hôm nay(realtime), và biểu đồ cơ giới các phương tiện trong bãi
STAFF_01	Check-In	Hoàn thành	Hệ thống gửi ảnh lên server hoạt động trơn chu và trả về được vị trí và khu vực khách đỗ.
STAFF_02	Check-out	Hoàn thành	Hệ thống gửi ảnh lên server hoạt động trơn tru và trả về hoá đơn cho khách hàng

3.2. Đánh giá kết quả thực nghiệm

Qua quá trình kiểm thử thực tế với các kịch bản sử dụng (Test Cases) đã đề ra, hệ thống đạt được các kết quả sau:

3.2.1. Kiểm thử Logic Đặt chỗ (Reservation Logic)

STT	Mô tả kịch bản	Dữ liệu đầu vào	Trạng thái
1	Người dùng đặt chỗ bằng cách tìm kiếm, chọn chỗ và thanh toán thành công	08:00 - 10:00, Slot A1(Slot đang trống)	Pass
2	Người dùng chọn lại khung giờ 8:00-10:00 slot A1 sẽ không còn hiển thị nữa	(08:00 - 10:00)	Pass
4	Hệ thống có kiểm tra các reservation quá hạn thanh toán cập nhập trạng thái về cancel	Đơn đặt chỗ với trạng thái pending quá 15 phút chưa thanh toán.	Pass

3.2.2. Kiểm thử Quy trình Check-in (AI & Vào bến)

STT	Mô tả kịch bản	Dữ liệu đầu vào	Trạng thái
1	Xe đã có đặt chỗ và đến check-in hệ thống hiển thị đúng vị trí spot đã đặt (nếu spot đó không bận)	Xe 30A-12345, vị trí A1	Pass
2	Xe vào check-in/ check-out, plate recognizer không nhận diện được. Hệ thống thông báo vui lòng chờ check-in/check-out lại		Pass
3	Xe chưa có đặt trước và đến theo hình thức vãng lai. Hệ thống tìm kiếm ghế vãng lai cho khách	Xe 29B-99999	Pass
4	Xe đến nhưng slot đã đầy. Hệ thống hiển thị slot đã đầy.	Xe 29B-99999	Pass

3.2.3. Kiểm thử Quy trình Check-out & Tính tiền

STT	Mô tả kịch bản	Dữ liệu đầu vào	Trạng thái
1	Xe đã đặt chỗ trước và check-out đúng giờ thì số tiền phải trả là 0 đồng	Xe ra lúc 10:00 (đúng giờ đặt). Đã trả qua VNPay.	Pass
2	Xe có đặt trước nhưng ra quá giờ đặt. Hệ thống tính phí phạt	Xe đặt đến 10:00 nhưng ra lúc 11:00 (Quá 1 tiếng).	Pass
3	Xe đến trước thời gian đặt chỗ và thời gian đỗ nằm trong reservation no-show. Hệ thống tự trừ tiền cho chỗ reservation no-show đó	Xe đến sớm (bị tính là vãng lai), nhưng User có 1 đơn "No-show" cũ cùng khung giờ.	Pass

3.2.4. Kiểm thử Quy trình quản lý của quản trị viên

STT	Mô tả kịch bản	Dữ liệu đầu vào	Trạng thái
1	Thêm nhân viên mới(chưa tồn tại trong cơ sở dữ liệu)	Name: Hà Gia Minh Date: 13/09/2003 Username: JaeMin Password: 123456	Pass
2	Thêm nhân viên mới(đã tồn tại trong cơ sở dữ liệu)	Name: Hà Gia Minh Date: 13/09/2003 Username: JaeMin Password: 123456	Pass
3	Xoá tài khoản nhân viên(xoá mềm)	Username: JaeMin	Pass
4	Xem danh sách nhân viên đã bị xoá. Kết quả mong đợi là Username: JaeMin không được hiển thị		Pass
5	Khôi phục tài khoản nhân viên	Username: JaeMin	Pass
6	Update tài khoản nhân viên	Username: JaeMin Name: Hà Gia Bình Date: 10/03/2010 Password: 12345789	Pass
7	Quản trị viên khóa tài khoản nhân viên, nhân viên sẽ không thể đăng nhập hoặc nếu nhân viên đang sử dụng sẽ bị thoát ra khỏi hệ thống luôn.	Username: JaeMin	Pass

STT	Mô tả kịch bản	Dữ liệu đầu vào	Trạng thái
8	Xoá một khu vực đỗ, hệ thống thực hiện xoá mềm để đảm bảo tính toàn vẹn của dữ liệu	Vị trí: A1	Pass

3.2.5. Kiểm thử hiệu năng và real-time

STT	Tên kịch bản	Kết quả mong đợi	Trạng thái
1	Cập nhật trạng thái thời gian thực	Ô tô vào check in và vị trí đỗ xe đó chuyển từ Xanh sang Đỏ ngay lập tức trong trang giao diện của admin và trang của người dùng số lượng đỗ xe ô tô giảm đi 1 (không cần F5).	Pass
2	Tốc độ nhận diện AI	Thời gian trung bình từ lúc gửi đến lúc nhận kết quả biển số nhỏ hơn 5 giây	Pass

CHƯƠNG 4: TỔNG KẾT

4.1. Kết luận

Sau quá trình nghiên cứu, phân tích và thực hiện đồ án "Xây dựng hệ thống quản lý bãi đỗ xe thông minh", nhóm thực hiện đã hoàn thành việc xây dựng một giải pháp phần mềm hoàn chỉnh, đáp ứng được các mục tiêu ban đầu đề ra.

4.1.1. Kết quả đạt được

Về mặt kỹ thuật và chức năng, hệ thống đã đạt được những thành quả cụ thể sau:

Xây dựng thành công hệ thống Web trọn gói: Đồ án đã phát triển hoàn thiện hai phân hệ chính: Trang dành cho Khách hàng (Client Side) sử dụng Next.js và Trang dành cho nhân viên giúp quản lý toàn diện hoạt động của bãi xe và hỗ trợ việc checkin/checkout của xe.

Giải quyết bài toán nghiệp vụ phức tạp: hệ thống xử lý tốt logic đặt chỗ (Reservation), ngăn chặn triệt để tình trạng đặt trùng lịch (Conflict Booking) thông qua việc kiểm tra khung giờ chặt chẽ.

Tự động hóa quy trình vận hành: áp dụng thành công Cron Job để tự động quét và hủy các đơn đặt quá hạn, giúp tối ưu hóa tài nguyên bãi đỗ mà không cần can thiệp thủ công.

Thông báo cho khách hàng khi sắp hết thời gian đỗ và tự động checkin nếu 2 reservation của khách liền nhau

Ứng dụng công nghệ thời gian thực (Real-time): Tích hợp thành công Socket.io giúp người dùng không cần không cần login mà vẫn xem được thực trạng slot hiện tại của bãi đỗ xe giúp tiết kiệm và quản trị viên có thể xem trạng thái của các slot trong bãi đỗ xe(realtime) giúp thuận tiện cho việc quản lý.

Tích hợp đa dịch vụ:

Hệ thống tích hợp api thứ ba(<https://platerecognizer.com/>) để xác định được biểu số và loại, điều này giúp giảm lược quá trình checkin/checkout truyền thống, giúp tiết kiệm cho khách hàng và tăng trải nghiệm người dùng

Hệ thống tích hợp cloudinary: để lưu trữ ảnh xe khi checkin/checkout

4.1.2. Những điểm hạn chế

Phụ thuộc vào dịch vụ bên thứ ba: Chức năng nhận diện biển số sử dụng API miễn phí của Plate Recognizer nên độ trễ còn phụ thuộc vào đường truyền internet và giới hạn số lượt request trong ngày. Chưa tự xây dựng được model AI (như YOLO) chạy offline để tăng tốc độ xử lý.

Thiếu hụt phần cứng IoT thực tế: Đồ án hiện tại mới chỉ dừng lại ở mức độ phần mềm quản lý. Chưa tích hợp các cảm biến siêu âm (Ultrasonic Sensors) tại từng vị trí đỗ thực tế để phát hiện xe đỗ sai vị trí so với đơn đặt trên phần mềm.

Quy mô thử nghiệm: Hệ thống mới được triển khai và kiểm thử trên môi trường Localhost, chưa được đánh giá hiệu năng chịu tải (Load Testing) khi có hàng nghìn truy cập đồng thời trên môi trường Cloud Server thực tế.

4.2. Hướng phát triển trong tương lai

Để hệ thống trở nên hoàn thiện hơn và có khả năng ứng dụng thực tiễn cao, các hướng phát triển tiếp theo được đề xuất bao gồm:

- Phát triển ứng dụng di động (Mobile App): Xây dựng phiên bản ứng dụng gốc (Native App) trên nền tảng Android/iOS sử dụng React Native hoặc Flutter. Tích hợp định vị GPS để dẫn đường tài xế từ vị trí hiện tại đến đúng ô đỗ xe đã đặt trong bãi.
- Tích hợp hệ thống IoT sâu rộng: Kết nối phần mềm với hệ thống phần cứng thực tế (Camera IP, Cảm biến vị trí, Barie tự động, Bảng LED hiển thị số chỗ trống) thông qua giao thức MQTT để tạo thành một hệ thống IoT hoàn chỉnh (Smart Parking IoT System).
- Nâng cấp trí tuệ nhân tạo (AI): Tự huấn luyện mô hình nhận diện biển số riêng (Custom Model) để chạy trực tiếp trên Server (Edge Computing), giảm sự phụ thuộc vào API bên ngoài và tăng tốc độ xử lý nhận diện xuống dưới 1 giây.
- Tối ưu hóa kinh doanh: Phát triển thuật toán giá linh hoạt (Dynamic Pricing) - tự động tăng giá vé vào giờ cao điểm hoặc giảm giá vào giờ thấp điểm để điều tiết lưu lượng xe và tối đa hóa doanh thu.

TÀI LIỆU THAM KHẢO

- [1] Trần Đình Quê. Phân tích và thiết kế hệ thống thông tin. Học viện Công nghệ Bưu chính Viễn thông. 2018
- [2] Node.js Foundation. (2023). *Node.js v18.x Documentation*.
<https://nodejs.org/docs/latest-v18.x/api/>
- [3] Expressjs.com. (2023). *Express - Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>
- [4] Sequelize.org. (2023). *Sequelize v6 Reference Manual - Promise-based Node.js ORM*. [<https://sequelize.org/>
- [5] MySQL. (2023). *MySQL 8.0 Reference Manual*.
<https://dev.mysql.com/doc/refman/8.0/en/>
- [6] Nextjs.org. (2023). *Next.js Documentation*. <https://nextjs.org/docs>
- [7] Socket.IO. (2023). *Socket.IO Documentation - Real-time Application Framework*.
<https://socket.io/docs/v4/>
- [8].Momentjs.com.(2023).*Moment Timezone Documentation*.
<https://momentjs.com/timezone/docs/>
- [9] *Plate Recognizer API Documentation*. <https://docs.platerecognizer.com/>
- [10] VNPay. (2023). *Tài liệu tích hợp cổng thanh toán VNPay (Sandbox)*.
<https://sandbox.vnpayment.vn/apis/>
- [11] Cloudinary. (2023). *Cloudinary Node.js SDK Documentation*. [Online].
Available: https://cloudinary.com/documentation/node_integration